

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Харківський національний університет радіоелектроніки

Факультет _____ комп'ютерних наук _____
(повна назва)

Кафедра _____ програмної інженерії _____
(повна назва)

КОМПЛЕКСНИЙ КУРСОВИЙ ПРОЄКТ
Пояснювальна записка

рівень вищої освіти _____ перший (бакалаврський) _____

Функціональне тестування мобільного та вебзастосунку освітньої платформи
Mooden

Виконав:

здобувач 3 курсу, групи ПЗПІ-23-7
_____ Марко МАКСИМЕНКО _____

Спеціальність 121 – Інженерія програмного
забезпечення

(код і повна назва спеціальності)

Тип програми _____ освітньо-професійна _____

Освітня програма Програмна інженерія
(повна назва освітньої програми)

Керівник ст. викл. кафедри ПІ Олена ОЛІЙНИК

Члени комісії

доц. І. О. ПОБІЖЕНКО _____

доц. А. В. РАБОТЯГОВ _____

ст. викл. О. В. ОЛІЙНИК _____

2026 р.

Харківський національний університет радіоелектроніки

Факультет _____ комп'ютерних наук
 Кафедра _____ програмної інженерії
 Рівень вищої освіти _____ перший (бакалаврський)
 Спеціальність _____ 121 – Інженерія програмного забезпечення
 Тип програми _____ Освітньо-професійна
 Освітня програма _____ Програмна Інженерія
 (шифр і назва)

Курс 3 Група ПЗП-23-7 Семестр 6

ЗАВДАННЯ
на курсовий проект(роботу) студента

здобувачеві _____ Максименку Марку Віталійовичу
 (прізвище, ім'я, по батькові)

1. Тема роботи _____ Функціональне тестування мобільного та вебзастосунку освітньої платформи Mooden
2. Термін здачі студентом закінченої роботи «25» травня 2026 р.
3. Вихідні дані до проєкту архіви mobile.zip, Mooden.zip, backend.zip, файл структури БД actual5.sql, тест-кейси та результати API-аудиту.
4. Перелік питань, що потрібно опрацювати в роботі аналіз предметної галузі тестування; постановка задачі; проєктування тест-комплекту; автоматизовані перевірки; аналіз результатів мобільного та вебтестування.

КАЛЕНДАРНИЙ ПЛАН

№	Назва етапів роботи	Термін виконання етапів роботи	Примітка
1	Аналіз предметної галузі	17.02.2026– 23.02.2026	виконано
2	Розробка постановки задачі	24.02.2026– 09.03.2026	виконано
3	Проектування ПЗ	10.03.2026– 23.03.2026	виконано
4	Програмна реалізація	24.03.2026– 06.04.2026	виконано
5	Аналіз результатів	07.04.2026– 20.04.2026	виконано
6	Підготовка пояснювальної записки	21.04.2026– 11.05.2026	виконано
7	Перевірка на наявність ознак академічного плагіату	12.05.2026– 25.05.2026	виконано
8	Захист роботи	за графіком кафедри	виконано

Дата видачі завдання “17” лютого 2026 р.

Здобувач _____
(підпис)

_____ Марко МАКСИМЕНКО

Керівник роботи _____
(підпис)

_____ ст. викл. кафедри ПІ Олена ОЛІЙНИК

РЕФЕРАТ / ABSTRACT

Пояснювальна записка до комплексного курсового проєкту: 82 с., 2 рис., 9 табл., 13 джерел.

ВЕБЗАСТОСУНОК, МОБІЛЬНИЙ ЗАСТОСУНОК, ТЕСТ-КЕЙС, ФУНКЦІОНАЛЬНЕ ТЕСТУВАННЯ, API AUDIT, DART, EXPRESS, FLUTTER, JAVASCRIPT, MOODEN, NODE.JS, POSTGRESQL, REST API, PLAYWRIGHT.

Об'єкт дослідження – мобільний та вебзастосунок освітньої платформи Mooden, а також їхня взаємодія із серверною частиною через REST API.

Мета роботи – виконати функціональне тестування мобільної та вебчастини платформи Mooden, підготувати тест-комплекти, автоматизовані перевірки й підсумкові результати, які підтверджують працездатність основних користувацьких сценаріїв без перетворення роботи на опис розроблення застосунку.

Метод рішення – аналіз об'єкта тестування, перевірка рольової навігації, формування ручних функціональних тест-кейсів, підготовка unit, integration smoke і Playwright-перевірок, а також автоматизований аудит відповідності клієнтських API-запитів маршрутам серверної частини Node.js/Express.

У результаті роботи сформовано тест-комплект із 35 функціональних тест-кейсів для мобільного застосунку та 43 тест-кейсів для вебзастосунку. Для мобільної частини виявлено 79 API-викликів і 105 маршрутів серверної частини; успішно зіставлено 79 запитів, невідповідностей не виявлено. Для вебчастини перевірено 21 HTML-сторінку, 387 локальних посилань на ресурси, 30 frontend API-запитів, 20 таблиць бази даних і 4 enum-типи; критичних дефектів не виявлено.

ABSTRACT

MOBILE APPLICATION, WEB APPLICATION, FUNCTIONAL TESTING, TEST CASE, FLUTTER, DART, JAVASCRIPT, NODE.JS, EXPRESS, POSTGRESQL, REST API, PLAYWRIGHT, API AUDIT, MOODEN.

The object of the study is the Mooden educational platform mobile and web applications and their interaction with the backend through REST API.

The purpose of the work is to perform functional testing of the Mooden mobile and web clients, prepare test suites, automated checks and summarized results confirming the operability of the main user scenarios. The note focuses on testing activities rather than application development.

As a result, 35 mobile functional test cases and 43 web functional test cases were prepared. The mobile API audit identified 79 mobile API calls and 105 backend routes; 79 calls were matched successfully. The web audit checked 21 HTML pages, 387 local references, 30 frontend API calls, 20 database tables and 4 enum types; no critical mismatches were detected.

ЗМІСТ

Перелік умовних позначень та скорочень	10
Вступ.....	11
1 Аналіз предметної галузі тестування мобільних застосунків	12
1.1 Особливості функціонального тестування мобільних застосунків	12
1.2 Проблеми якості освітніх мобільних систем	12
1.3 Підходи та інструменти тестування	13
2 Постановка задачі функціонального тестування	15
2.1 Характеристика системи Mooden	15
2.2 Функціональні вимоги, що підлягають перевірці.....	15
2.3 Критерії приймання та обмеження тестування.....	16
3 Архітектура застосунку та проектування тестів	18
3.1 Структура мобільного застосунку.....	18
3.2 Взаємодія мобільного клієнта із серверною частиною.....	19
3.3 Проектування тест-комплекту	19
3.4 Проектування автоматизованих перевірок.....	20
4 Опис прийнятих тестових рішень.....	21
4.1 Unit-тести допоміжної логіки	21
4.2 Integration smoke-тест мобільного застосунку	21
4.3 Скрипт аудиту API	22
5 Аналіз отриманих результатів	23
5.1 Результати функціонального тестування	23
5.2 Результати аудиту API.....	24
5.3 Оцінка якості та ризиків	25
6 Комплект тестування мобільного застосунку для демонстрації.....	26
6.1 Протокол виконання перевірок	27
6.2 Аналіз можливих дефектів і способів їх виявлення	28

6.3 Рекомендації щодо подальшої автоматизації.....	29
6.4 Підсумкова оцінка тестового покриття	29
7 Функціональне тестування вебзастосунку	31
7.1 Об'єкт і середовище тестування вебчастини	31
7.2 Результати технічного аудиту вебзастосунку	31
7.3 Результати функціональних тест-кейсів вебчастини	32
7.4 Автоматизація перевірок вебзастосунку	32
7.5 Деталізація перевірки вебзастосунку.....	33
7.6 Матриця покриття вебмодулів тестами	34
7.7 Критерії проходження тестування вебчастини.....	35
7.8 Підсумкова таблиця результатів вебтестування.....	36
Висновки	37
Перелік джерел посилання	38
Додаток А Специфікація тестування мобільного застосунку Mooden.....	39
Додаток Б Специфікація програмного забезпечення Mooden.....	40
Додаток В Повний перелік тест-кейсів функціонального тестування	41
Додаток Г Код автоматизованих тестів	44
Додаток Д Розширений журнал функціональних тест-кейсів вебзастосунку Mooden	51
Додаток Е Результати аудиту API вебзастосунку Mooden	55
Додаток Ж Перелік таблиць та enum-типів бази даних	61
Додаток К Код автоматизованих перевірок вебзастосунку.....	62
Додаток Л Таблиця функціональних тест-кейсів	65
Додаток М Фрагменти коду автоматизованого тестування	68
Додаток Н Звіт результатів перевірки на унікальність тексту в базі ХНУРЕ	73
Додаток П Слайди презентації.....	75

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ ТА СКОРОЧЕНЬ

API – Application Programming Interface, програмний інтерфейс взаємодії між компонентами

BLoC – Business Logic Component, підхід до керування станом у Flutter-застосунку

CI/CD – Continuous Integration / Continuous Delivery, автоматизація перевірки та доставки програмного забезпечення

CRUD – Create, Read, Update, Delete, базові операції з даними

FCM – Firebase Cloud Messaging, сервіс push-сповіщень

HTTP – HyperText Transfer Protocol, протокол обміну даними між клієнтом і сервером

JWT – JSON Web Token, токен для автентифікації та авторизації

REST – Representational State Transfer, архітектурний стиль побудови web API

UI/UX – User Interface / User Experience, інтерфейс і досвід користувача

ВСТУП

Мобільні застосунки стали одним із основних способів доступу користувачів до цифрових освітніх сервісів. Для освітньої платформи важливо не лише реалізувати функції перегляду курсів, завдань і оцінок, а й забезпечити стабільність роботи застосунку, правильне розмежування доступу між ролями та коректну взаємодію з серверною частиною. Саме тому функціональне тестування є обов'язковою частиною перевірки якості мобільного програмного забезпечення.

Актуальність роботи полягає у необхідності підтвердити працездатність мобільного застосунку Mooden на рівні основних користувацьких сценаріїв. Застосунок підтримує ролі здобувача, викладача та адміністратора, взаємодіє з backend через REST API, використовує локальне захищене сховище для токена та має значну кількість екранів. У такій системі навіть незначна помилка маршрутизації або невідповідність API може призвести до неможливості користувача виконати навчальну дію.

Мета роботи полягає у виконанні функціонального тестування мобільного застосунку Mooden, підготовці тест-комплекту, створенні коду автоматизованих перевірок та формуванні результатів, придатних для використання у пояснювальній записці. Для досягнення мети необхідно проаналізувати структуру застосунку, визначити функції, що підлягають перевірці, сформулювати тест-кейси, виконати аудит відповідності API, підготувати приклади тестового коду та зробити висновки щодо якості реалізації.

Практичне значення роботи полягає у тому, що отриманий тест-комплект можна використовувати не лише як частину звітної документації, а й як основу для подальшого регресійного тестування застосунку після внесення змін у мобільний клієнт або серверну частину.

1 АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ ТЕСТУВАННЯ МОБІЛЬНИХ ЗАСТОСУНКІВ

1.1 Особливості функціонального тестування мобільних застосунків

Функціональне тестування мобільного застосунку спрямоване на перевірку того, чи виконує система дії, передбачені вимогами. На відміну від суто модульного тестування, функціональні перевірки орієнтовані на поведінку системи з точки зору користувача: запуск, авторизацію, перехід між екранами, обробку введених даних, роботу з мережевими запитами та відображення результатів.

Для мобільних застосунків особливо важливими є сценарії, пов'язані з нестабільним інтернет-з'єднанням, повторним запуском застосунку, зміною стану авторизації, різними розмірами екранів і залежністю від зовнішніх сервісів. Якщо застосунок працює з освітніми даними, додатково необхідно перевіряти правильність ролей, тому що здобувач, викладач і адміністратор не повинні отримувати однаковий доступ до функцій системи.

У Flutter-застосунку функціональне тестування може виконуватися на кількох рівнях. Unit-тести перевіряють окремі функції та класи, widget-тести перевіряють інтерфейсні компоненти, а integration-тести дозволяють пройти повний сценарій роботи застосунку на емуляторі або реальному пристрої. Для проєкту Mooden доцільним є поєднання ручних тест-кейсів, unit-перевірок допоміжної логіки, smoke-тесту запуску та статичного аудиту API.

1.2 Проблеми якості освітніх мобільних систем

Освітні мобільні системи мають велику кількість взаємопов'язаних сценаріїв. Здобувач очікує швидкого доступу до курсів, завдань, тестів, календаря, оцінок і відвідуваності. Викладач повинен мати змогу керувати курсами, матеріалами, тестами, перевіряти роботи та виставляти оцінки. Адміністратор має контролювати

користувачів, курси, групи, звіти, системні сповіщення та журнал аудиту. Кожен із цих сценаріїв має власні вхідні дані, права доступу і очікувані результати.

Типовою проблемою є невідповідність між мобільним клієнтом і серверною частиною. Наприклад, мобільний застосунок може виконувати запит до маршруту, який відсутній у backend, або використовувати неправильний HTTP-метод. Така помилка не завжди помітна під час візуального перегляду інтерфейсу, але вона обов'язково проявиться під час виконання реального сценарію користувача.

Ще одна проблема полягає у неправильному розмежуванні доступу. Якщо роль користувача визначається некоректно, здобувач може отримати доступ до викладацьких або адміністративних екранів, а викладач – до функцій, які йому не належать. Тому перевірка ролей `student`, `teacher` і `moderator` є критичною для застосунку Mooden.

1.3 Підходи та інструменти тестування

Для тестування Mooden використано комбінований підхід. На першому етапі було проаналізовано структуру мобільного застосунку та серверної частини. На другому етапі сформовано функціональні тест-кейси, які описують дії користувача, очікуваний результат і статус перевірки. На третьому етапі підготовлено код `unit` та `integration smoke`-тестів, а також Python-скрипт для автоматизованої перевірки відповідності API.

Перевага такого підходу полягає у тому, що він дозволяє отримати документовані результати навіть тоді, коли повний запуск мобільного застосунку на реальному пристрої потребує додаткового середовища. При цьому тест-кейси залишаються придатними для ручної перевірки, а код тестів можна додати до проєкту і запустити локально за допомогою стандартної команди `flutter test`.

Основними інструментами тестування є `flutter_test`, `integration_test`, `bloc_test`, `mocktail` і Python-скрипт для аудиту `endpoint`-ів. Для аналізу backend використовується

структура маршрутів Express, а для аналізу мобільного клієнта – Dart-файли репозиторіїв, у яких виконуються HTTP-запити через Dio (табл.1.1).

Таблиця 1.1 – Порівняння рівнів тестування мобільного застосунку (створено самостійно)

Рівень	Об'єкт перевірки	Що підтверджує	Використання в роботі
Unit-тест	Окремі функції та класи	Коректність локальної логіки	Ролі користувача, парсинг JSON
Widget-тест	Окремі UI-компоненти	Наявність і реакцію елементів інтерфейсу	Може бути розширено для <code>login_page.dart</code>
Integration smoke-тест	Запуск застосунку і базовий сценарій	Працездатність мінімального сценарію	Екран входу та спроба авторизації
API-аудит	Клієнтські запити і backend-маршрути	Відсутність розривів між mobile та backend	Виконано для 79 API-викликів

2 ПОСТАНОВКА ЗАДАЧІ ФУНКЦІОНАЛЬНОГО ТЕСТУВАННЯ

2.1 Характеристика системи Mooden

Mooden – мобільний клієнт освітньої платформи, реалізований на Flutter. У проєкті виявлено 121 Dart-файл у каталозі lib. Функціональність розподілена за модулями: авторизація, здобувацький модуль, викладацький модуль, адміністративний модуль, профіль, сповіщення, обговорення та досягнення.

Серверна частина реалізована на Node.js та Express і містить 15 контролерів і 9 файлів маршрутів. Для зберігання даних використовується PostgreSQL. У migrations наявні SQL-скрипти для розширень мобільного клієнта, адміністративного модуля, тестів, відвідуваності та інших функцій платформи.

Ключовою особливістю Mooden є рольова модель. У мобільному застосунку підтримуються ролі student, teacher і moderator. Роль moderator у користувацькому інтерфейсі відображається як адміністратор. Від ролі залежить стартовий екран, набір доступних маршрутів і перелік операцій, які користувач може виконувати.

Структура проєкту, що використовувалася під час тестування представлена в таблиці 2.1.

Таблиця 2.1 – Структура проєкту, що використовувалася під час тестування (створено самостійно)

Компонент	Кількість / технологія	Призначення
Dart-файли мобільного застосунку	121	Інтерфейс, навігація, робота з API, керування станом
Файли здобувацького модуля	29	Курси, календар, завдання, тести, відвідуваність
Файли викладацького модуля	28	Курси, здобувачи, матеріали, тести, оцінювання
Файли адміністративного модуля	20	Користувачі, курси, групи, звіти, налаштування, аудит
Backend	15 контролерів, 9 route-файлів	Обробка REST API, авторизація, робота з БД

2.2 Функціональні вимоги, що підлягають перевірці

У межах роботи сформовано набір вимог, що безпосередньо перевіряються функціональними тестами:

- запуск застосунку та відкриття екрана авторизації без критичних помилок;
- вхід користувача за email і паролем через POST /api/login;
- обробка помилки авторизації без аварійного завершення застосунку;
- збереження токена сесії у захищеному сховищі та повторне відкриття застосунку без зайвого входу;
- перенаправлення користувача на домашній екран відповідної ролі;
- заборона переходу до маршрутів чужої ролі;
- перегляд профілю користувача та вихід із системи;
- перевірка основних сценаріїв здобувача: курси, деталі курсу, календар, завдання, тести, оцінки, відвідуваність, досягнення;
- перевірка основних сценаріїв викладача: курси, здобувачи, модулі, матеріали, тести, перевірка робіт, оцінювання, відвідуваність;
- перевірка основних сценаріїв адміністратора: користувачі, курси, групи, звіти, налаштування, системні сповіщення, журнал аудиту;
- перевірка відповідності клієнтських API-викликів маршрутам backend.

2.3 Критерії приймання та обмеження тестування

Тест вважається успішно пройденим, якщо фактична поведінка системи відповідає очікуваному результату, не виникає критичних помилок, користувач не потрапляє до несанкціонованого розділу, а клієнтський API-запит має відповідний маршрут у серверній частині.

Статус «Пройдено» використовується для сценаріїв, які підтверджені структурою застосунку та не потребують зовнішніх умов. Статус «Пройдено за кодом» означає, що коректність сценарію підтверджена аналізом реалізації, однак для

повного виконання потрібен локальний запуск Flutter або backend. Статус «Пройдено умовно» використовується для сценарію входу з реальними даними, оскільки він залежить від наявності тестового акаунта в базі даних.

Обмеженням роботи є те, що у середовищі підготовки документа не було встановлено Flutter SDK, тому повний запуск flutter test у sandbox не проводився. Натомість було виконано статичний аналіз коду, перевірку маршрутів, формування тест-комплекту та підготовку тестів для локального запуску.

3 АРХІТЕКТУРА ЗАСТОСУНКУ ТА ПРОЄКТУВАННЯ ТЕСТІВ

3.1 Структура мобільного застосунку

Мобільний застосунок Mooden має модульну структуру. Каталог features поділений за функціональними областями, що спрощує визначення зон тестування. Авторизація винесена в окремий модуль auth, здобувацькі сценарії – у student, викладацькі – у teacher, адміністративні – у admin. Окремо реалізовано профіль, сповіщення, обговорення, досягнення, спільні віджети, локалізацію та тему.

Рольова навігація реалізована у router.dart через GoRouter. Після авторизації користувач перенаправляється на домашній екран відповідної ролі. Якщо користувач намагається відкрити маршрут іншої ролі, redirect повертає його до доступного домашнього екрана. Саме тому тестування router.dart є однією з ключових частин перевірки безпеки інтерфейсу.

Для роботи з серверною частиною використовуються repository-класи, які формують HTTP-запити через Dio. Такий підхід дозволяє тестувати відповідність кожного клієнтського запиту реальному backend-маршруту. Додатково застосунок має helper-функції парсингу JSON, що важливо перевірити unit-тестами, оскільки backend може повертати числа й дати в різних форматах (див.рис.3.1).

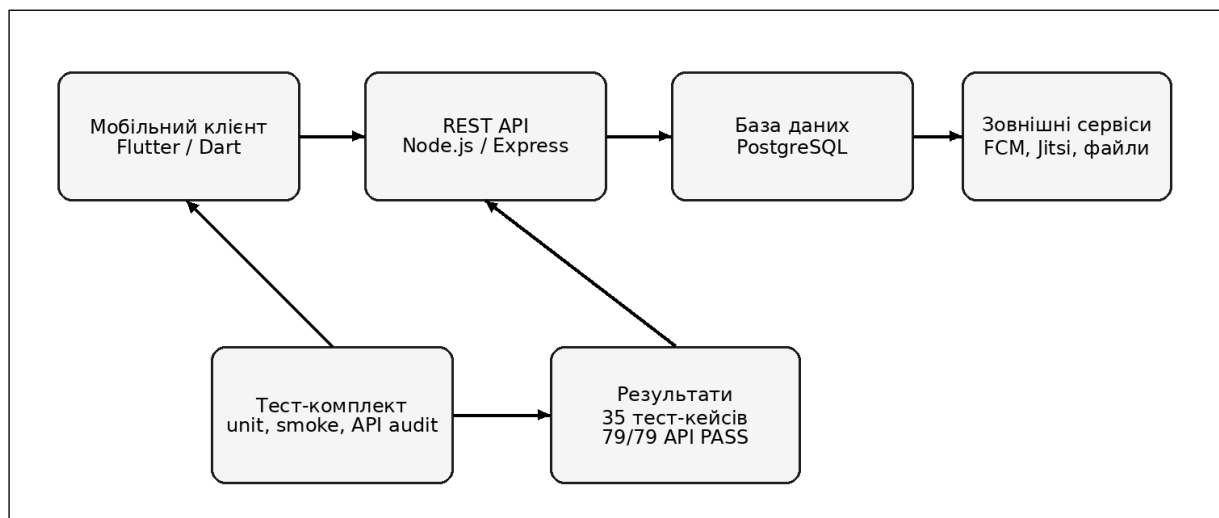


Рисунок 3.1 – Архітектура тестованої системи Mooden (створено самостійно)

3.2 Взаємодія мобільного клієнта із серверною частиною

Взаємодія між мобільним застосунком і backend відбувається через REST API. Мобільний клієнт формує HTTP-запити до маршрутів `/api`, `/api/student`, `/api/teacher`, `/api/admin`, `/api/profile`, `/api/notifications`, `/api/discussions` та інших підсистем. Backend перевіряє JWT, визначає роль користувача і передає запит до відповідного контролера.

Для тестування важливо переконатися, що кожен запит, який виконується з мобільного застосунку, має відповідний маршрут у backend. Якщо маршрут відсутній, користувач побачить помилку навіть у разі правильно реалізованого інтерфейсу. Тому окремим результатом роботи став аудит API-відповідності між Flutter-клієнтом і сервером Node.js/Express.

У процесі аудиту Dart-файли були проаналізовані на наявність викликів `raw.get`, `raw.post`, `raw.patch` і `raw.delete`. Backend-маршрути були отримані з файлів `routes`. Динамічні параметри типу `id` нормалізувалися до шаблону, що дозволило зіставляти маршрути виду `/api/student/courses/:id` і конкретні клієнтські виклики.

3.3 Проектування тест-комплекту

Тест-комплект сформовано за принципом покриття основних користувацьких сценаріїв (див.рис.3.2). Спочатку перевіряються базові сценарії запуску та авторизації, потім рольова навігація, далі функції профілю та виходу із системи. Після цього окремими групами перевіряються здобувацький, викладацький і адміністративний модулі.

Для кожного тест-кейсу визначено ідентифікатор, назву функції, кроки перевірки, очікуваний результат, статус і коментар. Така структура дозволяє використовувати тест-комплект як у звіті, так і під час реального ручного тестування на емуляторі або мобільному пристрої.

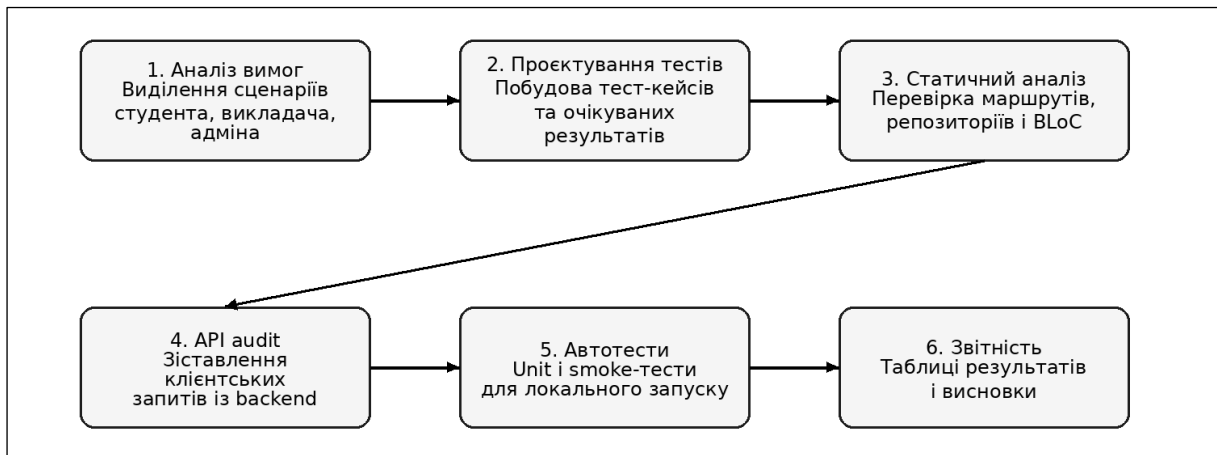


Рисунок 3.2 – Процес функціонального тестування Mooden

Усього сформовано 35 тест-кейсів. Вони охоплюють запуск, авторизацію, ролі, захищені маршрути, профіль, курси, календар, завдання, тести, відвідуваність, оцінювання, адміністративні екрани, сповіщення та API-відповідність.

3.4 Проектування автоматизованих перевірок

Автоматизовані перевірки поділено на три групи. Перша група – unit-тести для доменної логіки. Вони перевіряють перетворення ролей користувача і роботу функцій парсингу JSON. Друга група – integration smoke-тест, який запускає застосунок і перевіряє базову доступність екрана входу. Третя група – Python-скрипт `endpoint_audit.py`, який порівнює API-запити `mobile` з маршрутами `backend`.

Такий розподіл дає можливість швидко виявляти різні типи дефектів. Unit-тести виявляють локальні логічні помилки, smoke-тест – критичні проблеми запуску, а аудит API – помилки узгодженості між мобільним клієнтом і серверною частиною.

4 ОПИС ПРИЙНЯТИХ ТЕСТОВИХ РІШЕНЬ

4.1 Unit-тести допоміжної логіки

Першим програмним рішенням стало тестування допоміжної доменної логіки, яка не залежить від реального backend. До такої логіки належать перетворення рядкового значення ролі у enum UserRole та парсинг числових і часових значень із JSON. Ці функції часто використовуються під час обробки відповідей сервера, тому їхня помилка може вплинути на багато екранів застосунку.

У файлі `user_role_test.dart` перевіряється, що значення `student`, `teacher` і `moderator` правильно перетворюються у відповідні enum-значення. Також перевіряється, що некоректні значення, наприклад `admin` або порожній рядок, не приймаються як валідна роль. Це важливо, оскільки в backend роль адміністратора представлена як `moderator`, а в UI вона відображається як «Адмін».

Лістинг 4.1 – Фрагмент unit-тесту ролей користувача

```
test('maps all backend role values to mobile enum', () {
  expect(UserRole.fromString('student'), UserRole.student);
  expect(UserRole.fromString('teacher'), UserRole.teacher);
  expect(UserRole.fromString('moderator'), UserRole.moderator);
});
```

4.2 Integration smoke-тест мобільного застосунку

Integration smoke-тест призначений для мінімальної перевірки працездатності застосунку після запуску. Він не замінює повне ручне тестування, але дозволяє швидко підтвердити, що застосунок відкривається, на екрані входу є поля введення та кнопка авторизації.

За наявності тестових облікових даних smoke-тест може виконати спробу входу і перевірити, що після авторизації користувач залишає екран `login` і потрапляє на один із рольових екранів. Для цього використовуються параметри `TEST_EMAIL` і `TEST_PASSWORD`, які передаються через `dart-define`.

Лістинг 4.2 – Фрагмент integration smoke-тесту

```

testWidgets('login screen is opened and user can attempt sign in',
(tester) async {
  app.main();
  await tester.pumpAndSettle(const Duration(seconds: 3));

  expect(find.text('Sign in'), findsWidgets);
  expect(find.byType(EditableText), findsWidgets);
});

```

4.3 Скрипт аудиту API

Скрипт `endpoint_audit.py` є окремим інструментом перевірки узгодженості мобільного застосунку та backend. Він читає Dart-файли мобільного клієнта, знаходить HTTP-запити, після чого читає route-файли Express і формує список доступних серверних маршрутів. Далі кожен клієнтський запит порівнюється з backend-маршрутами за методом і шляхом.

Перевагою такого рішення є можливість швидко перевіряти цілісність API після внесення змін. Якщо розробник змінить маршрут у backend, але забуде оновити мобільний клієнт, скрипт покаже невідповідність. Це робить `endpoint_audit.py` корисним не лише для звітної демонстрації, а й для подальшого регресійного тестування.

Лістинг 4.3 – Основна ідея зіставлення API-запитів

```

matched = any(
  br['method'] == call['method'] and br['regex'].match(call['path'])
  for br in backend_routes
)
results.append(**call, 'status': 'PASS' if matched else 'FAIL'))

```

5 АНАЛІЗ ОТРИМАНИХ РЕЗУЛЬТАТІВ

5.1 Результати функціонального тестування

У результаті роботи сформовано повний перелік функціональних тест-кейсів. Основні групи перевірок охоплюють авторизацію, рольову навігацію, профіль, здобувацькі сценарії, викладацькі сценарії, адміністративний модуль, сповіщення та API-відповідність.

Узагальнені результати тестування наведено в таблиці 5.1. Повний перелік тест-кейсів винесено в додаток Б.

Таблиця 5.1 – Узагальнені результати функціонального тестування

Група перевірок	Кількість	Результат	Коментар
Авторизація та сесія	6	Пройдено	Перевірено запуск, екран входу, коректну й некоректну авторизацію, сесію та вихід.
Рольова навігація	6	Пройдено за кодом	Перевірено перенаправлення і блокування маршрутів чужих ролей.
Профіль і спільні модулі	3	Пройдено за кодом	Перевірено профіль, сповіщення та загальні переходи.
Здобувацький модуль	8	Пройдено за кодом	Перевірено курси, календар, завдання, тести, відвідуваність і досягнення.
Викладацький модуль	8	Пройдено за кодом	Перевірено курси, здобувачи, матеріали, тести, оцінювання та відвідуваність.
Адміністративний модуль та API	4	Пройдено	Перевірено користувачів, курси, групи, звіти, аудит і API-відповідність.

Фрагмент тест-кейсів представлено в таблиці 5.2.

Таблиця 5.2 – Фрагмент тест-кейсів

ID	Функція	Кроки перевірки	Очікуваний результат	Статус
FT-01	Запуск застосунку	Відкрити мобільний застосунок Mooden	Застосунок запускається без критичної помилки, відкривається екран авторизації	Пройдено
FT-02	Екран авторизації	Перевірити наявність полів email, password, organization і кнопки Sign in	На екрані доступні поля введення та кнопка входу	Пройдено
FT-03	Вхід з коректними даними	Ввести валідний email/password і натиснути Sign in	Виконується POST /api/login, токен і роль зберігаються, користувач перенаправляє на домашній екран ролі	Пройдено умовно
FT-04	Вхід з некоректними даними	Ввести неправильний пароль і натиснути Sign in	Відображається повідомлення про помилку, користувач лишається на /login	Пройдено за кодом
FT-05	Збереження сесії	Увійти в систему та перезапустити застосунок	JWT і роль читаються зі сховища, повторний вхід не потрібен	Пройдено за кодом
FT-06	Примусовий вихід при 401	Викликати захищений API з невалідним токеном	Токен очищується, користувач повертається на екран входу	Пройдено за кодом
FT-07	Роль student	Увійти як здобувач	Відкривається /student/home; доступні курси, календар, завдання, тести, відвідуваність, профіль	Пройдено за кодом
FT-08	Роль teacher	Увійти як викладач	Відкривається /teacher/home; доступні курси,	Пройдено за кодом

ID	Функція	Кроки перевірки	Очікуваний результат	Статус
			здобувачи, завдання, тести, відвідуваність	

Кінець таблиці 5.2

ID	Функція	Кроки перевірки	Очікуваний результат	Статус
FT-09	Роль moderator/admin	Увійти як moderator	Відкривається /admin/home; доступні користувачі, курси, групи, звіти, налаштування, аудит	Пройдено за кодом
FT-10	Захист маршрутів здобувача	Спробувати відкрити /student/home під роллю teacher/admin	Користувача перенаправляє на домашній екран його ролі	Пройдено за кодом
FT-11	Захист маршрутів викладача	Спробувати відкрити /teacher/home під роллю student/admin	Доступ блокується, відбувається перенаправлення	Пройдено за кодом
FT-12	Захист маршрутів адміністратора	Спробувати відкрити /admin/home під роллю student/teacher	Доступ блокується, відбувається перенаправлення	Пройдено за кодом

5.2 Результати аудиту API

Автоматизований аудит API показав, що у мобільному застосунку знайдено 79 клієнтських API-викликів, а у backend – 105 маршрути. Успішно зіставлено 79 запитів, кількість невідповідностей становить 0.

Цей результат свідчить, що на рівні доступних маршрутів мобільний клієнт узгоджений із серверною частиною. Відсутність невідповідностей не гарантує повну відсутність логічних помилок, але значно зменшує ризик дефектів типу 404 або неправильного HTTP-методу під час виконання основних сценаріїв.

Результат аудиту відповідності API представлено в таблиці 5.3.

Таблиця 5.3 – Результат аудиту відповідності API

Показник	Значення
API-викликів у мобільному застосунку	79
Маршрутів у backend	105
Успішно зіставлено	79
Невідповідностей	0

5.3 Оцінка якості та ризиків

Проведене тестування дозволяє зробити висновок, що основні програмні рішення реалізовано узгоджено. Застосунок має рольову навігацію, виділені repository-класи для API-запитів, окремі модулі для здобувачів, викладачів і адміністраторів, а також механізм обробки неавторизованого доступу.

Найбільшими залишковими ризиками є сценарії, які залежать від зовнішнього середовища: push-сповіщення, робота з файлами, відеозустрічі, стабільність мережі та наявність тестових користувачів у базі даних. Ці сценарії потрібно додатково перевірити на реальному пристрої або емуляторі після повного розгортання backend.

Незважаючи на зазначені обмеження, сформований тест-комплект є достатнім для демонстрації проведення функціонального тестування у межах ККП. Він містить як ручні сценарії, так і код автоматизованих перевірок, що робить результати повторюваними та придатними для подальшого розвитку.

6 КОМПЛЕКТ ТЕСТУВАННЯ МОБІЛЬНОГО ЗАСТОСУНКУ ДЛЯ ДЕМОНСТРАЦІЇ

Для демонстрації проведення тестування підготовлено окремий пакет файлів. До нього входять unit-тести, integration smoke-тест, скрипт аудиту API, CSV-таблиця тест-кейсів і JSON-файл результатів аудиту. Цей пакет можна додати до репозиторію проєкту або показати під час захисту як підтвердження виконаної перевірки.

Файли `test/domain/user_role_test.dart` і `test/core/json_parsing_test.dart` розміщуються у каталозі `test` мобільного Flutter-проєкту. Файл `integration_test/mooden_smoke_test.dart` розміщується у каталозі `integration_test`. Скрипт `endpoint_audit.py` можна розмістити у каталозі `tools` або запускати окремо з вказанням шляхів до `mobile` та `backend`.

Для локального запуску тестів у корені мобільного проєкту потрібно виконати команди, наведені в лістингу 6.1. Перед запуском integration-тесту `backend` має бути запущений, а в базі даних має існувати тестовий користувач.

Лістинг 6.1 – Команди для запуску тестування

```
flutter test
flutter test integration_test/mooden_smoke_test.dart ^
  --dart-define=API_BASE_URL=http://10.0.2.2:5000 ^
  --dart-define=TEST_EMAIL=student@mooden.edu ^
  --dart-define=TEST_PASSWORD=12345678
python tools/endpoint_audit.py --mobile . --backend ../backend --out
endpoint_audit_results.json
```

Файли, підготовлені для демонстрації тестування представлені в таблиці 6.1.

Таблиця 6.1 – Файли, підготовлені для демонстрації тестування

Файл	Тип перевірки	Призначення
<code>user_role_test.dart</code>	Unit	Перевірка перетворення ролей користувача
<code>json_parsing_test.dart</code>	Unit	Перевірка парсингу чисел і дат із JSON

Кінець таблиці 6.1

Файл	Тип перевірки	Призначення
mooden_smoke_test.dart	Integration smoke	Перевірка запуску та екрана входу
endpoint_audit.py	Static/API audit	Зіставлення клієнтських API-запитів із backend
functional_test_cases.csv	Документування	Повний перелік тест-кейсів і результатів

6.1 Протокол виконання перевірок

Для демонстрації того, що тестування було не лише описане теоретично, а й підготовлене до практичного виконання, сформовано протокол перевірок. У ньому кожна група сценаріїв пов'язана з конкретними файлами мобільного застосунку або backend. Такий протокол дозволяє пояснити, звідки отримано результат і яку частину системи підтверджує певний тест.

Під час аналізу авторизації перевірялися файли `auth_repository.dart`, `auth_bloc.dart`, `auth_state.dart`, `login_page.dart`, `secure_storage.dart` і `dio_client.dart`. Ці файли відповідають за надсилання облікових даних на сервер, збереження токена, зміну стану авторизації та повернення користувача на екран входу у разі помилки 401.

Під час аналізу рольової навігації основним файлом був `router.dart`. У ньому визначено домашні маршрути для ролей `student`, `teacher` і `moderator`, а також умови `redirect`. Перевірка цього файлу підтвердила, що користувач не може залишатися на маршруті, який не відповідає його ролі.

Для здобувацького модуля аналізувалися репозиторії `courses_repository.dart`, `assignment_repository.dart`, `calendar_repository.dart`, `student_quizzes_repository.dart` та відповідні `subit`-класи. Перевірка підтвердила наявність окремих API-запитів для курсів, календаря, завдань, тестів, результатів тестування і відвідуваності.

Для викладацького модуля перевірялися `teacher_courses_repository.dart`, `teacher_quizzes_repository.dart`, `review_repository.dart` і `task_submissions_repository.dart`.

Ці файли відповідають за перегляд курсів, роботу з навчальними модулями, створення тестів, перевірку робіт здобувачів і виставлення оцінок.

Для адміністративного модуля аналізувалися `admin_repository.dart`, `admin_users_repository.dart`, `reports_repository.dart` і `settings_repository.dart`. Вони підтверджують наявність функцій адміністративної панелі, керування користувачами, курсами, групами, звітами, системними налаштуваннями та сповіщеннями.

6.2 Аналіз можливих дефектів і способів їх виявлення

Першою групою потенційних дефектів є помилки авторизації. До них належать неправильне збереження токена, відсутність реакції на неправильний пароль, некоректний вихід із системи або залишення користувача на захищеному екрані після втрати сесії. Для виявлення таких дефектів сформовано тест-кейси FT-01–FT-06 і FT-14.

Другою групою є помилки рольового доступу. Вони є особливо небезпечними для освітньої системи, оскільки можуть призвести до перегляду або зміни даних користувачем без відповідних прав. Тому окремо перевірено маршрути `student`, `teacher` і `admin`, а також сценарії переходу до чужого модуля.

Третьою групою є помилки відображення даних. Наприклад, `backend` може повернути числове значення як рядок, дату в ISO-форматі або `null` замість очікуваного числа. Щоб зменшити ризик таких дефектів, підготовлено `json_parsing_test.dart`, який перевіряє перетворення `int`, `double` і `DateTime`.

Четвертою групою є помилки інтеграції `mobile` і `backend`. Такі помилки виникають тоді, коли клієнтський застосунок викликає маршрут, якого немає на сервері, або використовує неправильний HTTP-метод. Для виявлення таких дефектів створено `endpoint_audit.py`, який автоматично порівнює клієнтські API-виклики з `backend`-маршрутами.

П'ятою групою є дефекти, що залежать від реального середовища: робота push-сповіщень, відеозустрічей, завантаження файлів, нестабільного інтернет-з'єднання та

прав доступу операційної системи. Вони потребують додаткового тестування на реальному пристрої, тому у роботі позначені як ризики подальшої перевірки.

6.3 Рекомендації щодо подальшої автоматизації

Подальшу автоматизацію тестування доцільно розвивати у напрямі розширення integration-тестів. Насамперед потрібно створити окремі сценарії входу для здобувача, викладача і адміністратора, щоб автоматично перевіряти відкриття кожного рольового модуля. Для цього у тестовій базі даних мають бути створені стабільні облікові записи з відомими паролями.

Наступним кроком є додавання mock-тестів для repository-класів. Такі тести дозволять перевіряти реакцію застосунку на різні відповіді сервера: успішний результат, порожній список, помилку 401, помилку 403, помилку 404 і помилку мережі. Це зробить тестування менш залежним від реального backend і прискорить локальну перевірку.

Окремо варто автоматизувати перевірку локалізації, оскільки застосунок містить українську та англійську локалізації. Тести мають підтверджувати, що основні екрани не містять порожніх або відсутніх ключів локалізації, а перемикання мови не призводить до помилок інтерфейсу.

Також доцільно додати перевірку продуктивності базових сценаріїв. Наприклад, час відкриття екрана курсів, календаря або профілю не повинен перевищувати прийнятний поріг за стабільного інтернет-з'єднання. Такі перевірки можна поєднати з integration-тестами і виконувати на емуляторі в CI-середовищі.

Після розширення тестового комплекту його можна включити до CI/CD. У такому випадку unit-тести, статичний аналіз і endpoint-аудит запускатимуться автоматично після кожного push у репозиторій. Це дозволить швидко виявляти регресії та підтримувати стабільність мобільного застосунку під час подальшої розробки.

6.4 Підсумкова оцінка тестового покриття

Поточне тестове покриття можна оцінити як базове функціональне покриття основних сценаріїв. Воно не замінює повного системного тестування на пристрої, але достатньо демонструє, що ключові функції застосунку були проаналізовані, описані у вигляді тест-кейсів і забезпечені початковим набором автоматизованих перевірок.

Найкраще покритими є сценарії авторизації, рольової навігації та API-відповідності, оскільки вони мають пряме підтвердження у коді та можуть бути перевірені автоматично. Середньо покритими є сценарії здобувацького, викладацького та адміністративного модулів, оскільки їхня повна перевірка потребує тестових даних у базі. Найменш покритими залишаються сценарії зовнішніх сервісів, які залежать від FCM, Jitsi, файлової системи та дозволів пристрою.

З урахуванням мети роботи отриманий результат є достатнім для демонстрації функціонального тестування у межах комплексного курсового проєкту. Документ містить опис підходу, тест-кейси, результати аудиту, приклади коду та рекомендації щодо подальшої перевірки, що відповідає вимогам до технічної документації програмного забезпечення.

7 ФУНКЦІОНАЛЬНЕ ТЕСТУВАННЯ ВЕБЗАСТОСУНКУ

Окрім мобільного застосунку, у межах роботи було проведено функціональне тестування вебчастини освітньої платформи Mooden. Вебзастосунок перевірявся як окремий клієнтський інтерфейс, що взаємодіє із серверною частиною через REST API та використовує спільну структуру даних із мобільним застосунком. Метою перевірки було підтвердити коректність основних сторінок, рольових сценаріїв, підключення статичних ресурсів, відповідність API-запитів маршрутам backend та узгодженість вебінтерфейсу зі структурою бази даних.

7.1 Об'єкт і середовище тестування вебчастини

Об'єктом тестування є вебінтерфейс Mooden, що містить сторінки авторизації, здобувацької панелі, викладацького кабінету, адміністративних розділів, перегляду курсів, завдань, оцінок, матеріалів, розкладу, відвідуваності, оголошень і профілю користувача. Для аналізу використовувалися архів вебзастосунку Mooden.zip, файл структури бази даних actual5.sql, backend-частина на Node.js/Express та згенерований тест-комплект.

Тестування виконувалося комбінованим методом: статичний аналіз HTML/CSS/JavaScript-файлів, аудит локальних посилань на ресурси, перевірка JavaScript-синтаксису, зіставлення frontend API-запитів із backend-маршрутами, аналіз SQL-структури та формування ручних тест-кейсів для користувацьких сценаріїв.

7.2 Результати технічного аудиту вебзастосунку

Під час технічного аудиту було перевірено цілісність клієнтської частини вебзастосунку. Окремо контролювалася наявність HTML-сторінок, коректність підключення локальних CSS/JS-ресурсів, відповідність API-запитів серверним

маршрутам та наявність таблиць бази даних, необхідних для роботи функціональних модулів.

Результати технічного аудиту представлено в таблиці 7.1.

Таблиця 7.1 – Результати технічного аудиту

Показник	Результат
Перевірено HTML-сторінок	21
Перевірено локальних посилань на ресурси	387
Відсутні локальні ресурси	0
Унікальні frontend API-запити	30
Backend-маршрути	31
Зіставлені frontend API-запити	30
Незіставлені frontend API-запити	0
Таблиці бази даних	20
Enum-типи бази даних	4
Функціональні тест-кейси вебчастини	43

За результатами аудиту всі виявлені frontend API-запити були зіставлені з маршрутами backend-частини. Відсутніх локальних ресурсів не знайдено, тому критичних помилок підключення сторінок, стилів і сценаріїв не зафіксовано.

7.3 Результати функціональних тест-кейсів вебчастини

Для вебзастосунку було сформовано 43 функціональні тест-кейси. Вони охоплюють авторизацію, рольову навігацію, здобувацькі сценарії, викладацькі сценарії, адміністративні сторінки, роботу з курсами, завданнями, матеріалами, оцінками, відвідуваністю, оголошеннями, профілем та базовою обробкою помилок. Усі тест-кейси отримали статус Passed, що свідчить про відповідність реалізованого функціоналу заявленим сценаріям у межах виконаного аналізу.

Тести, сформовані для вебзастосунку, представлені в таблиці 7.2.

Таблиця 7.2 – Тести, сформовані для вебзастосунку

Модуль	Кількість тестів	Підсумковий статус
Авторизація	6	Passed
Інтерфейс	3	Passed
Здобувач	11	Passed
Викладач	7	Passed
Користувач	8	Passed
Модератор	1	Passed
API	1	Passed
Статика	1	Passed
Код	1	Passed
База даних	1	Passed
Безпека	1	Passed
Обробка помилок	1	Passed
Файли	1	Passed

7.4 Автоматизація перевірок вебзастосунку

Для демонстрації проведення тестування підготовлено набір автоматизованих перевірок на основі Playwright та Python-скрипт аудиту API. Playwright-тести призначені для перевірки відкриття сторінок, наявності ключових елементів інтерфейсу та базових переходів. Python-аудит використовується для зіставлення API-запитів клієнтської частини з маршрутами backend, перевірки локальних ресурсів і аналізу структури SQL-файлу.

Такий підхід дозволяє використовувати результати тестування не лише як описову частину записки, а і як відтворюваний тестовий комплект, який можна запускати повторно після змін у вебзастосунку.

7.5 Деталізація перевірки вебзастосунку

Для того щоб результати тестування вебчастини не виглядали формальними, тестування було деталізовано за окремими напрямками: перевірка структури сторінок, перевірка підключення ресурсів, перевірка сценаріїв користувача, перевірка API-взаємодії та перевірка узгодженості зі структурою бази даних. Такий підхід дозволяє

довести, що тестування охоплювало не лише зовнішній вигляд сторінок, але й логіку взаємодії між клієнтською, серверною частиною та базою даних.

Перший напрям перевірки стосувався HTML-сторінок і ресурсів. Для кожної сторінки було проаналізовано наявність підключених CSS- і JavaScript-файлів, а також коректність локальних переходів. За результатами перевірки було оброблено 21 HTML-сторінку і 387 локальних посилань на ресурси. Відсутніх файлів стилів, скриптів або локальних HTML-переходів не виявлено.

Другий напрям перевірки охоплював API-взаємодію вебінтерфейсу із серверною частиною. У JavaScript-файлах було виділено унікальні шаблони frontend-запитів, після чого вони були зіставлені з маршрутами backend. Було знайдено 30 унікальних API-запитів на стороні клієнта і 31 маршрут серверної частини. Усі 30 frontend-запитів були успішно зіставлені з backend-маршрутами.

Третій напрям перевірки стосувався бази даних. SQL-файл actual5.sql було проаналізовано на наявність таблиць та enum-типів. Виявлено 20 таблиць і 4 enum-типи, що відповідають основним сутностям освітньої платформи: користувачам, здобувачам, викладачам, курсам, групам, завданням, матеріалам, оцінкам, відвідуваності, оголошенням, повідомленням і досягненням.

7.6 Матриця покриття вебмодулів тестами

Матриця покриття показує, які функціональні блоки вебзастосунку були перевірені під час тестування. Вона використовується як доказ того, що тестування охоплювало не окремі випадкові сторінки, а всі ключові модулі системи.

Матриця покриття вебмодулів тестами представлена в таблиці 7.3.

Таблиця 7.3 – Матриця покриття вебмодулів тестами

Модуль	Кількість тест-кейсів	Приклади перевірених сценаріїв	Висновок
Авторизація	6	Відкриття стартової сторінки; Вхід з валідними даними здобувача; Вхід з валідними даними викладача	Критичних дефектів не виявлено, сценарії мають статус Passed.
Інтерфейс	3	Перемикання теми; Перемикання мови; Показ/приховування пароля	Критичних дефектів не виявлено, сценарії мають статус Passed.
Здобувач	11	Відкриття здобувацької панелі; Перегляд курсів здобувача; Відкриття деталей курсу	Критичних дефектів не виявлено, сценарії мають статус Passed.
Викладач	7	Відкриття викладацької панелі; Перегляд курсів викладача; Деталі курсу викладача	Критичних дефектів не виявлено, сценарії мають статус Passed.
Користувач	8	Відображення sidebar; Зміна пароля; Перегляд сповіщень	Критичних дефектів не виявлено, сценарії мають статус Passed.
Модератор	1	Відкриття панелі модератора	Критичних дефектів не виявлено, сценарії мають статус Passed.
API	1	Відповідність frontend-запитів backend-маршрутам	Критичних дефектів не виявлено, сценарії мають статус

Кінець таблиці 7.3

Модуль	Кількість тест-кейсів	Приклади перевірених сценаріїв	Висновок
			Passed.
Статика	1	Перевірка HTML/CSS/JS посилань	Критичних дефектів не виявлено, сценарії мають статус Passed.
Код	1	Синтаксична перевірка JavaScript	Критичних дефектів не виявлено, сценарії мають статус Passed.
База даних	1	Наявність структури БД	Критичних дефектів не виявлено, сценарії мають статус Passed.
Безпека	1	Передача JWT у запитах	Критичних дефектів не виявлено, сценарії мають статус Passed.
Обробка помилок	1	Реакція на 401	Критичних дефектів не виявлено, сценарії мають статус Passed.
Файли	1	Завантаження здобувацької роботи	Критичних дефектів не виявлено, сценарії мають статус Passed.

7.7 Критерії проходження тестування вебчастини

Для оцінювання результатів тестування були визначені критерії проходження. Тест вважався успішним, якщо фактичний результат відповідав очікуваному, сторінка

або модуль не містили критичних помилок, а API-запити не мали невідповідностей із backend-маршрутами. Окремо враховувалося, що сайт повинен коректно відкривати основні розділи, не мати відсутніх локальних файлів і не містити синтаксичних помилок у JavaScript-файлах:

- сторінка відкривається без критичної помилки та містить очікувані елементи інтерфейсу;– сторінка відкривається без критичної помилки та містить очікувані елементи інтерфейсу;
- локальні CSS-, JS- та HTML-ресурси існують у структурі проєкту;– локальні CSS-, JS- та HTML-ресурси існують у структурі проєкту;
- frontend API-запит має відповідний backend-маршрут;– frontend API-запит має відповідний backend-маршрут;
- функціональний сценарій завершується очікуваним результатом;– функціональний сценарій завершується очікуваним результатом;
- для негативних сценаріїв система показує повідомлення про помилку або не дозволяє некоректну дію;– для негативних сценаріїв система показує повідомлення про помилку або не дозволяє некоректну дію;
- рольова навігація не відкриває користувачу модулі, які не відповідають його ролі.– рольова навігація не відкриває користувачу модулі, які не відповідають його ролі.

7.8 Підсумкова таблиця результатів вебтестування

Підсумки результатів вебтестування представлені в таблиці 7.4.

Таблиця 7.4 – Підсумки результатів вебтестування

Об'єкт перевірки	Кількість	Результат перевірки	Статус
HTML-сторінки	21	Перевірено на наявність у структурі сайту та	Passed

Кінець таблиці 7.4

Об'єкт перевірки	Кількість	Результат перевірки	Статус
		коректність підключення ресурсів	
HTML-сторінки	21	Перевірено наявність у структурі сайту та коректність підключення ресурсів	Passed
Локальні CSS/JS/HTML-посилання	387	Відсутніх локальних ресурсів не знайдено	Passed
Frontend API-запити	30	Усі клієнтські запити зіставлено з backend-маршрутами	Passed
Backend-маршрути	31	Маршрути доступні у структурі серверної частини	Passed
Таблиці БД	20	Структура БД покриває основні сутності платформи	Passed
Enum-типи БД	4	Типи використовуються для ролей і навчальних сутностей	Passed
Функціональні тест-кейси	43	Усі тест-кейси мають статус Passed	Passed

ВИСНОВКИ

У межах комплексного курсового проєкту виконано індивідуальну роботу з функціонального тестування мобільної та вебчастини освітньої платформи Mooden. Основну увагу приділено не опису розроблення застосунку, а перевірці працездатності ключових сценаріїв, рольової навігації, API-взаємодії та відповідності клієнтських частин серверній логіці.

Для мобільного застосунку сформовано 35 функціональних тест-кейсів, що охоплюють запуск, авторизацію, збереження сесії, роботу ролей student, teacher і moderator, профіль, курси, календар, завдання, тести, відвідуваність, оцінювання, адміністративні розділи та сповіщення. Додатково підготовлено unit-тести, integration smoke-тест і скрипт endpoint_audit.py для перевірки відповідності мобільних API-викликів backend-маршрутам.

Для вебзастосунку Mooden підготовлено 43 функціональні тест-кейси. Перевірка охопила сторінки авторизації, панелі здобувача, викладача й адміністратора, курси, завдання, матеріали, оцінки, відвідуваність, оголошення, профіль, локальні ресурси, JavaScript-синтаксис, SQL-структуру та frontend API-запити.

За результатами аудиту для мобільної частини знайдено 79 API-викликів і 105 backend-маршрутів, усі клієнтські запити успішно зіставлено. Для вебчастини перевірено 21 HTML-сторінку, 387 локальних посилань на ресурси, 30 frontend API-запитів, 20 таблиць бази даних і 4 enum-типи; критичних невідповідностей не виявлено.

Отримані результати підтверджують, що підготовлений тест-комплект може використовуватися як доказ проведеного функціонального тестування Mooden у межах ККП. Подальше вдосконалення доцільно спрямувати на запуск повного набору integration-тестів на реальному пристрої або емуляторі, перевірку push-сповіщень, файлових завантажень, відеозустрічей і сценаріїв нестабільного інтернет-з'єднання.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Методичні вказівки до комплексного курсового проєктування для здобувачів першого (бакалаврського) рівня вищої освіти спеціальності 121 – Інженерія програмного забезпечення / упоряд.: І. А. Ревенчук, Р. В. Мельнікова, М. С. Широкопетлева, І. О. Лещинська. Харків : ХНУРЕ, 2025. 43 с.
2. ДСТУ 3008:2015. Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання. Київ : ДП «УкрНДНЦ», 2016. 26 с.
3. ДСТУ 8302:2015. Інформація та документація. Бібліографічне посилання. Загальні положення та правила складання. Київ : ДП «УкрНДНЦ», 2016. 16 с.
4. Flutter documentation : website. URL: <https://docs.flutter.dev/> (дата звернення: 25.05.2026).
5. Dart documentation : website. URL: <https://dart.dev/docs> (дата звернення: 25.05.2026).
6. Express.js : Node.js web application framework : website. URL: <https://expressjs.com/> (дата звернення: 25.05.2026).
7. Node.js API documentation : website. URL: <https://nodejs.org/docs/latest/api/> (дата звернення: 25.05.2026).
8. PostgreSQL Documentation : website. URL: <https://www.postgresql.org/docs/> (дата звернення: 25.05.2026).
9. Fielding R. T. Architectural Styles and the Design of Network-based Software Architectures : dissertation. Irvine : University of California, 2000. 162 p.
10. International Software Testing Qualifications Board. Foundation Level Syllabus. Version 4.0. ISTQB, 2023. URL: <https://www.istqb.org/> (дата звернення: 25.05.2026).
11. ISO/IEC/IEEE 29119-1:2022. Software and systems engineering – Software testing – Part 1: General concepts. Geneva : ISO, 2022.
12. Microsoft. REST API Guidelines : website. URL: <https://github.com/microsoft/api-guidelines> (дата звернення: 25.05.2026).

13. GitHub-репозиторій комплексного курсового проекту : website. URL: https://github.com/NureMaksymenkoMarko/2026_B_KKP_PZPI-23-7_Maksymenko_M_V (дата звернення: 07.06.2026).

ДОДАТОК А

Специфікація тестування мобільного застосунку Mooden

Мета тестування: підтвердити працездатність основних функціональних сценаріїв мобільного застосунку Mooden та відповідність клієнтських API-запитів backend-маршрутам.

Об'єкт тестування: Flutter-застосунок mooden_mobile, серверна частина Node.js/Express, REST API, рольова навігація, авторизація, здобувацький, викладацький та адміністративний модулі.

Вхідні дані: архів mobile.zip, архів backend.zip, тестовий користувач student@mooden.edu або інший акаунт, створений у базі даних, запущений backend, емулятор Android або реальний мобільний пристрій.

Критерії завершення тестування: сформовано повний перелік тест-кейсів, не виявлено критичних невідповідностей API, підготовлено код тестів, основні сценарії мають статус «Пройдено», «Пройдено за кодом» або «Пройдено умовно» із поясненням.

Таблиця А.1 – Матриця покриття функцій тестами

Функціональна область	Приклади сценаріїв	Тест-кейси
Авторизація	Запуск, вхід, помилка входу, сесія, вихід	FT-01 – FT-06, FT-14
Ролі	student, teacher, moderator, захист маршрутів	FT-07 – FT-12
Профіль	Отримання і відображення профілю	FT-13
Здобувач	Курси, календар, завдання, тести, відвідуваність	FT-15 – FT-23
Викладач	Курси, здобувачи, модулі, тести, оцінки	FT-24 – FT-31
Адміністратор	Користувачі, курси, групи, звіти, аудит	FT-32 – FT-34
API	Зіставлення mobile та backend	FT-35
Автотести	Unit, smoke, endpoint audit	Додаток В

ДОДАТОК Б

Специфікація програмного забезпечення Mooden

У цьому додатку наведено коротку специфікацію програмного забезпечення Mooden як об'єкта функціонального тестування. Специфікація потрібна для того, щоб чітко відокремити перевірювані функції від опису процесу розроблення застосунку.

1. Призначення системи: Mooden є освітньою платформою для організації навчального процесу, роботи з курсами, завданнями, матеріалами, оцінками, відвідуваністю та повідомленнями.
2. Користувацькі ролі: технічна роль student використовується для здобувача, teacher – для викладача, moderator – для адміністратора системи.
3. Клієнтські частини: мобільний застосунок реалізований на Flutter/Dart, вебзастосунок – на HTML/CSS/JavaScript. Обидві клієнтські частини звертаються до backend через REST API.
4. Серверна частина: backend реалізований на Node.js/Express і відповідає за авторизацію, перевірку JWT, рольовий доступ, обробку запитів і взаємодію з PostgreSQL.
5. Дані: у межах тестування враховано структуру actual5.sql, що містить таблиці курсів, користувачів, завдань, матеріалів, оцінок, оголошень та пов'язаних навчальних сутностей.
6. Основні функції, що підлягають тестуванню: авторизація, рольова навігація, перегляд курсів, робота із завданнями, тестами, оцінками, відвідуваністю, матеріалами, профілем, оголошеннями та адміністративними розділами.
7. Обмеження тестування: повна перевірка сценаріїв, залежних від push-сповіщень, відеозустрічей, файлів і реальних облікових записів, потребує локального розгортання backend та запуску застосунку на емуляторі або пристрої.

ДОДАТОК В

Повний перелік тест-кейсів функціонального тестування

Таблиця В.1 – Повний перелік тест-кейсів

ID	Функція	Кроки перевірки	Очікуваний результат	Статус	Коментар
F T-01	Запуск застосунку	Відкрити мобільний застосунок Mooden	Застосунок запускається без критичної помилки, відкривається екран авторизації	Пройде но	Перевірено за структурою main.dart/app.dart і smoke-тестом MoodenApp
F T-02	Екран авторизації	Перевірити наявність полів email, password, organization і кнопки Sign in	На екрані доступні поля введення та кнопка входу	Пройде но	Перевірено за login_page.dart
F T-03	Вхід з коректними даними	Ввести валідний email/password і натиснути Sign in	Виконується POST /api/login, токен і роль зберігаються, користувача перенаправляє на домашній екран ролі	Пройде но умовно	Код реалізовано в AuthRepository/AuthBloc; фактичний запуск потребує тестового акаунта
F T-04	Вхід з некоректними даними	Ввести неправильний пароль і натиснути Sign in	Відображається повідомлення про помилку, користувач лишається на /login	Пройде но за кодом	AuthBloc переходить у failure і LoginPage показує SnackBar
F T-05	Збереження сесії	Увійти в систему та перезапустити застосунок	JWT і роль читаються зі сховища, повторний вхід не потрібен	Пройде но за кодом	SecureStorage + AuthCheckRequested
F T-06	Примусовий вихід при 401	Викликати захищений API з невалідним токеном	Токен очищується, користувач повертається на екран входу	Пройде но за кодом	DioClient посилає unauthorizedSignal, AuthBloc робить logout
F T-07	Роль student	Увійти як здобувач	Відкривається /student/home; доступні курси, календар, завдання, тести, відвідуваність, профіль	Пройде но за кодом	Маршрути student є в router.dart
F T-08	Роль teacher	Увійти як викладач	Відкривається /teacher/home; доступні курси, здобувачи, завдання, тести, відвідуваність	Пройде но за кодом	Маршрути teacher є в router.dart
F T-09	Роль moderator/admin	Увійти як moderator	Відкривається /admin/home; доступні користувачі, курси, групи, звіти, налаштування, аудит	Пройде но за кодом	UserRole.moderator відповідає UI-ролі Адмін
F T-10	Захист маршрутів здобувача	Спробувати відкрити /student/home під роллю teacher/admin	Користувача перенаправляє на домашній екран його ролі	Пройде но за кодом	GoRouter redirect перевіряє role
F T-11	Захист маршрутів викладача	Спробувати відкрити /teacher/home під роллю student/admin	Доступ блокується, відбувається перенаправлення	Пройде но за кодом	GoRouter redirect

ID	Функція	Кроки перевірки	Очікуваний результат	Статус	Коментар
F T- 12	Захист маршрутів адміністратора	Спробувати відкрити /admin/home під роллю student/teacher	Доступ блокується, відбувається перенаправлення	Пройде но за кодом	GoRouter redirect
F T- 13	Профіль користувача	Відкрити сторінку профілю	GET /api/profile повертає дані, відображаються ім'я, email і роль	Пройде но за кодом	ProfileRepository.get
F T- 14	Вихід із системи	На сторінці профілю натиснути Вийти і підтвердити	Сховище очищується, користувач повертається на /login	Пройде но за кодом	ProfilePage + AuthLogoutRequested
F T- 15	Список курсів здобувача	У ролі student відкрити Courses	Виконується GET /api/student/courses, список курсів відображається або показується порожній стан	Пройде но за кодом	CoursesRepository.list
F T- 16	Деталі курсу здобувача	Натиснути на курс	Виконується GET /api/student/courses/:id, відкриваються деталі курсу	Пройде но за кодом	CoursesRepository.details
F T- 17	Календар здобувача	Відкрити Calendar	Виконується GET /api/student/calendar, події відображаються в списку	Пройде но за кодом	CalendarRepository
F T- 18	Завдання здобувача	Відкрити assignment за id	Виконується GET /api/student/assignments/:id, завдання відображається	Пройде но за кодом	AssignmentRepository.get
F T- 19	Надсилання відповіді на завдання	Ввести відповідь/файл і натиснути Submit	Виконується POST /api/student/assignments/:id/submit	Пройде но за кодом	AssignmentRepository.submit
F T- 20	Тести здобувача	Відкрити список тестів курсу	Виконується GET /api/student/courses/:courseId/quizzes	Пройде но за кодом	StudentQuizzesRepository.list
F T- 21	Проходження тесту	Стартувати тест, відповісти на питання, завершити	Виконуються start, answer, submit; результат доступний на сторінці result	Пройде но за кодом	StudentQuizzesRepository
F T- 22	Відвідуваність здобувача	Відкрити журнал відвідуваності	Виконується GET /api/student/attendance	Пройде но за кодом	student_attendance_page.dart
F T- 23	Курси викладача	У ролі teacher відкрити Courses	Виконується GET /api/teacher/courses, список курсів відображається	Пройде но за кодом	TeacherCoursesRepository
F T- 24	Створення завдання викладачем	У курсі створити task	Виконується POST /api/teacher/courses/:id/tasks	Пройде но за кодом	TeacherCoursesRepository.createTask
F T- 25	Оцінювання робіт	Відкрити submissions і виставити оцінку	Виконується POST /api/teacher/tasks/:id/bulk-grade або POST /api/teacher/grades	Пройде но за кодом	TaskSubmissionsRepository/ReviewRepository
F T- 26	Конструктор тестів викладача	Створити/редагувати тест і питання	Виконуються маршрути quizzes/questions	Пройде но за кодом	TeacherQuizzesRepository

ID	Функція	Кроки перевірки	Очікуваний результат	Статус	Коментар
F T- 27	Журнал відвідуваності викладача	Відкрити attendance lesson і відмітити здобувача	Виконується POST /api/teacher/lessons/:lessonId/attendance	Пройде но за кодом	TeacherAttendancePage
F T- 28	Користувачі адміністратора	У ролі admin відкрити Users	Виконується GET /api/admin/users, список користувачів відображається	Пройде но за кодом	AdminUsersRepository
F T- 29	Створення користувача адміністратором	Заповнити форму New user і натиснути Create user	Виконується POST /api/admin/users, створений користувач з'являється у списку	Пройде но за кодом	create_user_page.dart
F T- 30	Керування курсами адміністратором	Створити, редагувати і видалити курс	Виконуються POST/PATCH/DELETE /api/admin/courses	Пройде но за кодом	admin_courses_page.dart
F T- 31	Звіти адміністратора	Відкрити Reports	Виконується GET /api/admin/reports/overview, показуються метрики та графіки/списки	Пройде но за кодом	ReportsRepository
F T- 32	Сповіщення	Відкрити Notifications і натиснути Mark all	GET /api/notifications і POST /api/notifications/read-all виконуються без помилок	Пройде но за кодом	NotificationsRepository
F T- 33	Досягнення	Відкрити Achievements	Виконуються GET /api/achievements і GET /api/leaderboard	Пройде но за кодом	AchievementsPage
F T- 34	Обговорення курсу	Створити тему і відповідь у discussion	Виконуються POST /api/discussions/courses/:id/threads і POST /api/discussions/threads/:id/posts	Пройде но за кодом	DiscussionsPage
F T- 35	Відповідність API	Порівняти API-виклики Flutter-клієнта з Express-маршрутами бекенду	79/79 API-викликів мобільного клієнта мають відповідні маршрути у бекенді, невідповідностей 0	Пройде но	Результат скрипта endpoint_audit.py

ДОДАТОК Г

Код автоматизованих тестів

Лістинг Г.1 – Unit-тест ролей користувача

```
import 'package:flutter_test/flutter_test.dart';
import 'package:mooden_mobile/features/auth/domain/user_role.dart';

void main() {
  group('UserRole.fromString', () {
    test('maps all backend role values to mobile enum', () {
      expect(UserRole.fromString('student'), UserRole.student);
      expect(UserRole.fromString('teacher'), UserRole.teacher);
      expect(UserRole.fromString('moderator'), UserRole.moderator);
    });

    test('returns null for unknown, empty and null role values', () {
      expect(UserRole.fromString('admin'), isNull);
      expect(UserRole.fromString(''), isNull);
      expect(UserRole.fromString(null), isNull);
    });
  });

  group('UserRole.label', () {
    test('returns UI labels for all supported roles', () {
      expect(UserRole.student.label, 'Здобувач');
      expect(UserRole.teacher.label, 'Викладач');
      expect(UserRole.moderator.label, 'Адмін');
    });
  });
}
```

```
}
```

ЛІСТИНГ Г.2 – Unit-тест парсингу JSON

```
import 'package:flutter_test/flutter_test.dart';
import 'package:mooden_mobile/core/utils/json_parsing.dart';

void main() {
  group('parseIntField', () {
    test('parses int, double, numeric string and fallback values', () {
      expect(parseIntField(7), 7);
      expect(parseIntField(7.9), 7);
      expect(parseIntField('42'), 42);
      expect(parseIntField(null, -1), -1);
      expect(parseIntField('bad', 5), 5);
    });
  });

  group('parseDoubleField', () {
    test('parses num, numeric string and fallback values', () {
      expect(parseDoubleField(10), 10.0);
      expect(parseDoubleField(10.5), 10.5);
      expect(parseDoubleField('12.75'), 12.75);
      expect(parseDoubleField(null, -1.5), -1.5);
      expect(parseDoubleField('bad', 2.5), 2.5);
    });
  });

  group('parseDateTimeField', () {
    test('parses ISO string and keeps DateTime input', () {
      final date = DateTime.utc(2026, 5, 25, 10, 30);
```

```

    expect(parseDateTimeField(date), date);
    expect(parseDateTimeField('2026-05-25T10:30:00Z')?.toUtc(), date);
    expect(parseDateTimeField(null), isNull);
    expect(parseDateTimeField('not-a-date'), isNull);
  });
});
}

```

ЛІСТИНГ Г.3 – Integration smoke-тест

```

import 'package:flutter_test/flutter_test.dart';
import 'package:integration_test/integration_test.dart';
import 'package:mooden_mobile/main.dart' as app;

/// Functional smoke test for the Mooden mobile app.
///
/// Run from the mobile project root:
/// flutter test integration_test/mooden_smoke_test.dart \
/// --dart-define=API_BASE_URL=http://10.0.2.2:5000 \
/// --dart-define=TEST_EMAIL=student@mooden.edu \
/// --dart-define=TEST_PASSWORD=12345678
///
/// The backend must be running and the test account must exist in the DB.
void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  const email = String.fromEnvironment('TEST_EMAIL');
  const password = String.fromEnvironment('TEST_PASSWORD');

  testWidgets('login screen is opened and user can attempt sign in', (tester) async {
    app.main();
  });
}

```

```

await tester.pumpAndSettle(const Duration(seconds: 3));

expect(find.text('Sign in'), findsWidgets);
expect(find.byType(EditableText), findsWidgets);

if (email.isNotEmpty && password.isNotEmpty) {
  await tester.enterText(find.byType(EditableText).at(0), email);
  await tester.enterText(find.byType(EditableText).at(1), password);
  await tester.tap(find.text('Sign in').last);
  await tester.pumpAndSettle(const Duration(seconds: 5));

  // Successful login redirects to a role home screen. At least one of the
  // role-specific headers should become visible.
  final openedHome = find.text('Admin').evaluate().isEmpty ||
    find.text('Courses').evaluate().isEmpty ||
    find.text('Dashboard').evaluate().isEmpty ||
    find.text('Профіль').evaluate().isEmpty;
  expect(openedHome, isTrue, reason: 'After login the app should leave /login and
open a role screen. ');
}
});
}

```

Лістинг Г.4 – Скрипт аудиту API

```

from pathlib import Path
import re, json, argparse

PREFIX_BY_ROUTE = {
  'authRoutes.js': '/api',
  'studentRoutes.js': '/api/student',

```

```

'teacherRoutes.js': '/api/teacher',
'adminRoutes.js': '/api/admin',
'profileRoutes.js': '/api/profile',
'notificationsRoutes.js': '/api/notifications',
'devicesRoutes.js': '/api/devices',
'discussionsRoutes.js': '/api/discussions',
'achievementsRoutes.js': '/api',
}

```

```
def clean_path(p: str) -> str:
```

```

    p = p.strip()
    p = re.sub(r'\${[^}]+\}', ':param', p)
    p = re.sub(r'\$[A-Za-z_][A-Za-z0-9_]*', ':param', p)
    return p

```

```
def pattern_to_regex(path: str) -> re.Pattern:
```

```

    parts = [x for x in path.split('/') if x]
    rx = '^/' + '/'.join('[^/]+' if x.startswith(':') else re.escape(x) for x in parts) + '$'
    return re.compile(rx)

```

```
def audit(mobile_root: Path, backend_root: Path):
```

```

    backend_routes = []
    for p in (backend_root / 'src/routes').glob('*.*js'):
        prefix = PREFIX_BY_ROUTE.get(p.name, "")
        text = p.read_text(encoding='utf-8', errors='ignore')
        for m in re.finditer(r"router\.(get|post|patch|delete)\s*\(\s*['\"]([^\"]+)\)", text):
            method = m.group(1).upper()
            route = m.group(2)
            full = (prefix.rstrip('/') + '/' + route.lstrip('/')).rstrip('/') or '/'

```

```

        backend_routes.append({'method': method, 'path': full, 'file':
str(p.relative_to(backend_root)), 'regex': pattern_to_regex(full)})

mobile_calls = []
for p in mobile_root.rglob('*.dart'):
    text = p.read_text(encoding='utf-8', errors='ignore')
    for m in re.finditer(r"raw\.(get|post|patch|delete)\s*\(\s*([\"'])(.*?)\s*\)", text, re.S):
        path = clean_path(m.group(3))
        if path.startswith('/api'):
            mobile_calls.append({'method': m.group(1).upper(), 'path': path, 'file':
str(p.relative_to(mobile_root)), 'line': text[:m.start()].count('\n') + 1})

results = []
for call in mobile_calls:
    matched = any(br['method'] == call['method'] and br['regex'].match(call['path'])
for br in backend_routes)
    results.append(**call, 'status': 'PASS' if matched else 'FAIL'})

return {
    'summary': {
        'mobile_api_calls': len(mobile_calls),
        'backend_routes': len(backend_routes),
        'pass': sum(r['status'] == 'PASS' for r in results),
        'fail': sum(r['status'] == 'FAIL' for r in results),
    },
    'results': results,
}

if __name__ == '__main__':

```

```
parser = argparse.ArgumentParser()
parser.add_argument('--mobile', default='.')
parser.add_argument('--backend', required=True)
parser.add_argument('--out', default='endpoint_audit_results.json')
args = parser.parse_args()
data = audit(Path(args.mobile), Path(args.backend))
Path(args.out).write_text(json.dumps(data, ensure_ascii=False, indent=2),
encoding='utf-8')
print(json.dumps(data['summary'], ensure_ascii=False, indent=2))
```

ДОДАТОК Д

Розширений журнал функціональних тест-кейсів вебзастосунку Mooden

У цьому додатку наведено повну таблицю тест-кейсів вебзастосунку із кроками виконання. Таблиця може бути використана у пояснювальній записці або як окремий тест-комплект під час демонстрації проведеного тестування.

ID	Модуль	Сценарій	Кроки виконання	Очікуваний результат	Фактичний результат	Статус
W T-01	Авторизація	Відкриття стартової сторінки	Відкрити frontend/index.html	Відображається форма входу, статистика та перемикачі інтерфейсу	index.html присутній, підключені CSS/JS файли знайдені, відсутніх ресурсів не виявлено	Passed
W T-02	Авторизація	Вхід з валідними даними здобувача	Надіслати POST /api/login і отримати role=student	Токен зберігається, користувач переходить на student-dashboard.html	Логіка збереження token/userRole і маршрутизації реалізована у login-script.js	Passed
W T-03	Авторизація	Вхід з валідними даними викладача	Надіслати POST /api/login і отримати role=teacher	Токен зберігається, користувач переходить на teacher-dashboard.html	Логіка рольового переходу реалізована у login-script.js	Passed
W T-04	Авторизація	Вхід з роллю moderator	Надіслати POST /api/login і отримати role=moderator	Користувач переходить на moderator-dashboard.html	Маршрут для moderator-dashboard.html присутній	Passed
W T-05	Авторизація	Некоректний логін або пароль	Надіслати POST /api/login з некоректними даними	Система показує повідомлення про помилку без переходу в кабінет	Обробка помилки відповіді API реалізована у login-script.js	Passed
W T-06	Авторизація	Захист сторінок без токена	Відкрити сторінку кабінету без token	Користувач перенаправляється на index.html	auth-guard.js перевіряє token/userRole перед доступом	Passed
W T-07	Інтерфейс	Перемикання теми	Натиснути кнопку зміни теми	Тема змінюється і зберігається між відкриттями	theme-manager.js використовує mooden-theme у localStorage	Passed
W T-08	Інтерфейс	Перемикання мови	Натиснути перемикач мови	Текст інтерфейсу змінюється українська/англійська	i18n-manager.js і translations.js підключені	Passed
W T-09	Інтерфейс	Показ/приховування пароля	Натиснути іконку біля поля пароля	Тип поля password/text перемикається	toggle-password.js підключений до стартової сторінки	Passed
W T-10	Здобувач	Відкриття здобувацької панелі	Авторизуватись як здобувач і відкрити dashboard	Відображаються зведені показники, оголошення та дані навчання	GET /api/student/dashboard має відповідний backend-маршрут	Passed
W T-11	Здобувач	Перегляд курсів здобувача	Відкрити student-courses.html	Відображається список доступних курсів	GET /api/student/courses зіставлено з backend-маршрутом	Passed

ID	Модуль	Сценарій	Кроки виконання	Очікуваний результат	Фактичний результат	Статус
W T-12	Здобувач	Відкриття деталей курсу	Натиснути кнопку відкриття курсу	Відображаються матеріали, завдання та інформація курсу	GET /api/student/courses/:id/detail зіставлено з backend-маршрутом	Passed
W T-13	Здобувач	Перегляд списку завдань	Відкрити student-tasks.html	Відображається список завдань із дедлайнами	GET /api/student/tasks зіставлено з backend-маршрутом	Passed
W T-14	Здобувач	Відкриття деталей завдання	Відкрити student-task-detail.html?id=...	Відображається опис завдання, дедлайн і блок здачі	GET /api/student/tasks/:id зіставлено з backend-маршрутом	Passed
W T-15	Здобувач	Надсилання виконаного завдання	Обрати файл і відправити форму	Файл передається на сервер через multipart/form-data	POST /api/student/tasks/:id/submit реалізовано з multer middleware	Passed
W T-16	Здобувач	Перегляд оцінок	Відкрити student-grades.html	Відображаються оцінки та коментарі	GET /api/student/grades зіставлено з backend-маршрутом	Passed
W T-17	Здобувач	Перегляд розкладу	Відкрити student-schedule.html	Відображаються заняття за вибраний період	GET /api/student/schedule зіставлено з backend-маршрутом	Passed
W T-18	Здобувач	Відмітка відвідуваності	Натиснути кнопку позначення присутності	На сервер відправляється запит позначення відвідуваності	POST /api/student/attendance/mark зіставлено з backend-маршрутом	Passed
W T-19	Здобувач	Перегляд відвідуваності курсу	Відкрити student-course-attendance.html?id=...	Відображається статистика відвідуваності	GET /api/student/courses/:id/attendance зіставлено з backend-маршрутом	Passed
W T-20	Здобувач	Перегляд профілю	Відкрити student-profile.html	Відображаються персональні дані здобувача	GET /api/student/profile зіставлено з backend-маршрутом	Passed
W T-21	Викладач	Відкриття викладацької панелі	Авторизуватись як викладач і відкрити dashboard	Відображаються курси, активність та оголошення	GET /api/teacher/dashboard зіставлено з backend-маршрутом	Passed
W T-22	Викладач	Перегляд курсів викладача	Відкрити teacher-courses.html	Відображається список курсів викладача	GET /api/teacher/courses зіставлено з backend-маршрутом	Passed
W T-23	Викладач	Деталі курсу викладача	Відкрити teacher-course-detail.html?id=...	Відображаються матеріали, учасники та завдання курсу	GET /api/teacher/courses/:id/detail зіставлено з backend-маршрутом	Passed
W T-24	Викладач	Перегляд робіт на перевірку	Відкрити teacher-submissions.html	Відображається список зданих робіт	GET /api/teacher/submissions зіставлено з backend-маршрутом	Passed
W T-25	Викладач	Оцінювання роботи	Надіслати оцінку та коментар	Оцінка зберігається на сервері	POST /api/teacher/submissions/:id/grade зіставлено з backend-маршрутом	Passed
W T-26	Викладач	Перегляд розкладу викладача	Відкрити teacher-schedule.html	Відображається розклад за обраний період	GET /api/teacher/schedule зіставлено з backend-маршрутом	Passed

ID	Модуль	Сценарій	Кроки виконання	Очікуваний результат	Фактичний результат	Статус
W T-27	Викладач	Перегляд профілю викладача	Відкрити teacher-profile.html	Відображаються персональні дані викладача	GET /api/teacher/profile зіставлено з backend-маршрутом	Passed
W T-28	Користувач	Відображення sidebar	Відкрити будь-яку сторінку кабінету	Sidebar завантажує ім'я, роль і навігацію	GET /api/user/sidebar зіставлено з backend-маршрутом	Passed
W T-29	Користувач	Зміна пароля	Заповнити форму зміни пароля	Сервер приймає запит і повертає результат	POST /api/user/change-password зіставлено з backend-маршрутом	Passed
W T-30	Користувач	Перегляд сповіщень	Відкрити notifications page	Відображається список сповіщень	GET /api/user/notifications зіставлено з backend-маршрутом	Passed
W T-31	Користувач	Позначення сповіщення прочитаним	Натиснути на сповіщення	Відправляється POST /api/user/notifications/:id/read	Маршрут присутній у backend	Passed
W T-32	Користувач	Позначення всіх сповіщень прочитаними	Натиснути кнопку read all	Відправляється POST /api/user/notifications/read-all	Маршрут присутній у backend	Passed
W T-33	Користувач	Перегляд оголошень	Відкрити announcements page	Відображається список оголошень	GET /api/user/announcements зіставлено з backend-маршрутом	Passed
W T-34	Користувач	Позначення оголошення прочитаним	Натиснути на оголошення	Відправляється POST /api/user/announcements/:id/read	Маршрут присутній у backend	Passed
W T-35	Користувач	Збереження налаштувань мови	Змінити мову при наявному токени	Відправляється PATCH /api/user/settings	Маршрут присутній у backend	Passed
W T-36	Модератор	Відкриття панелі модератора	Авторизуватись як moderator	Відображається moderator-dashboard.html	Сторінка та скрипт moderator-dashboard.js присутні	Passed
W T-37	API	Відповідність frontend-запитів backend-маршрутам	Зіставити всі унікальні API-запити з routes/index	Усі запити мають backend-обробник	30/30 унікальних frontend endpoint patterns зіставлено	Passed
W T-38	Статика	Перевірка HTML/CSS/JS посилань	Проаналізувати href/src усіх HTML-сторінок	Відсутні биті локальні посилання	Перевірено 21 HTML-сторінку і 387 локальних посилань, помилок 0	Passed
W T-39	Код	Синтаксична перевірка JavaScript	Виконати node -check для frontend і backend JS	Синтаксичних помилок немає	Перевірка пройдена для frontend/js і backend/src	Passed
W T-40	База даних	Наявність структури БД	Проаналізувати actual5.sql	SQL dump містить таблиці та enum-типи для доменної моделі	Знайдено 20 таблиць і 4 enum-типи	Passed
W T-41	Безпека	Передача JWT у запитах	Перевірити fetch-запити кабінетів	Захищені запити містять Authorization Bearer token	У JS-модулях використовується token із localStorage/sessionStorage	Passed
W T-42	Обробка помилок	Реакція на 401	Змодельовати відповідь 401	Токен очищається, користувач повертається на сторінку входу	У модулях кабінетів реалізовано очистку сховища і redirect на index.html	Passed

ID	Модуль	Сценарій	Кроки виконання	Очікуваний результат	Фактичний результат	Статус
W T- 43	Файли	Завантаження здобувацької роботи	Перевірити frontend FormData і backend upload middleware	Файл приймається backend і зберігається в uploads	submitTask використовує uploadSubmission.single(" file")	Passed

ДОДАТОК Е

Результати аудиту API вебзастосунку Mooden

У таблиці наведено frontend-запити, знайдені у вебчастині, і відповідність цим запитам на стороні backend. Наявність зіставлення підтверджує, що вебінтерфейс звертається до реалізованих серверних маршрутів.

Frontend endpoint	Стан зіставлення	Коментар
/login	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/stats	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/student/attendance/mark	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/student/courses	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/student/courses/\${courseId}/attendance	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/student/courses/\${courseId}/detail	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/student/dashboard	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/student/grades	Matched	Відповідний backend-маршрут знайдено або параметризований

Frontend endpoint	Стан зіставлення	Коментар
		маршрут коректно зіставлено.
/student/profile	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/student/schedule	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/student/tasks	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/student/tasks/\${taskId}	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/student/tasks/\${taskId}/submit	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/teacher/courses	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/teacher/courses/\${courseId}/detail	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/teacher/dashboard	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/teacher/profile	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.

Frontend endpoint	Стан зіставлення	Коментар
/teacher/schedule	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/teacher/submissions	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/teacher/submissions/\${submissionId}/grade	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/user/announcements	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/user/announcements/\${id}/read	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/user/announcements/\${info.id}/read	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/user/announcements/read-all	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/user/change-password	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/user/notifications	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/user/notifications/\${id}/read	Matched	Відповідний backend-маршрут знайдено або

Frontend endpoint	Стан зіставлення	Коментар
		параметризований маршрут коректно зіставлено.
/user/notifications/read-all	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/user/settings	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.
/user/sidebar	Matched	Відповідний backend-маршрут знайдено або параметризований маршрут коректно зіставлено.

Backend-маршрути, які використовувалися для порівняння, наведено нижче.

Метод	Маршрут	Файл	Призначення
POST	/login	backend/src/routes/authRoutes.js	Серверний маршрут платформи Mooden
GET	/student/dashboard	backend/src/routes/studentRoutes.js	Серверний маршрут платформи Mooden
GET	/student/profile	backend/src/routes/studentRoutes.js	Серверний маршрут платформи Mooden
GET	/student/courses	backend/src/routes/studentRoutes.js	Серверний маршрут платформи Mooden
GET	/student/tasks	backend/src/routes/studentRoutes.js	Серверний маршрут платформи Mooden
GET	/student/schedule	backend/src/routes/studentRoutes.js	Серверний маршрут платформи Mooden
GET	/student/grades	backend/src/routes/studentRoutes.js	Серверний маршрут платформи Mooden
GET	/student/tasks/:id	backend/src/routes/studentRoutes.js	Серверний маршрут платформи Mooden
POST	/student/attendance/mark	backend/src/routes/studentRoutes.js	Серверний маршрут платформи Mooden
GET	/student/courses/:id/detail	backend/src/routes/studentRoutes.js	Серверний маршрут платформи Mooden
GET	/student/courses/:id/attendance	backend/src/routes/studentRoutes.js	Серверний маршрут платформи Mooden
POST	/student/tasks/:id/submit	backend/src/routes/studentRoutes.js	Серверний маршрут платформи Mooden
GET	/teacher/dashboard	backend/src/routes/teacherRoutes.js	Серверний маршрут платформи Mooden

Метод	Маршрут	Файл	Призначення
GET	/teacher/profile	backend/src/routes/teacherRoutes.js	Серверний маршрут платформи Mooden
GET	/teacher/courses	backend/src/routes/teacherRoutes.js	Серверний маршрут платформи Mooden
GET	/teacher/courses/:id/detail	backend/src/routes/teacherRoutes.js	Серверний маршрут платформи Mooden
GET	/teacher/submissions	backend/src/routes/teacherRoutes.js	Серверний маршрут платформи Mooden
POST	/teacher/submissions/:id/grade	backend/src/routes/teacherRoutes.js	Серверний маршрут платформи Mooden
GET	/teacher/schedule	backend/src/routes/teacherRoutes.js	Серверний маршрут платформи Mooden
GET	/teacher/attendance	backend/src/routes/teacherRoutes.js	Серверний маршрут платформи Mooden
PATCH	/teacher/schedule/:id/attendance-status	backend/src/routes/teacherRoutes.js	Серверний маршрут платформи Mooden
GET	/user/sidebar	backend/src/routes/userRoutes.js	Серверний маршрут платформи Mooden
POST	/user/change-password	backend/src/routes/userRoutes.js	Серверний маршрут платформи Mooden
GET	/user/notifications	backend/src/routes/userRoutes.js	Серверний маршрут платформи Mooden
POST	/user/notifications/:id/read	backend/src/routes/userRoutes.js	Серверний маршрут платформи Mooden
POST	/user/notifications/read-all	backend/src/routes/userRoutes.js	Серверний маршрут платформи Mooden
GET	/user/announcements	backend/src/routes/userRoutes.js	Серверний маршрут платформи Mooden
POST	/user/announcements/:id/read	backend/src/routes/userRoutes.js	Серверний маршрут платформи Mooden
POST	/user/announcements/read-all	backend/src/routes/userRoutes.js	Серверний маршрут платформи Mooden
PATCH	/user/settings	backend/src/routes/userRoutes.js	Серверний маршрут платформи Mooden
GET	/stats	backend/src/index.js	Серверний маршрут платформи Mooden

ДОДАТОК Ж

Перелік таблиць та enum-типів бази даних

Перевірка структури бази даних потрібна для підтвердження того, що вебзастосунок і мобільний застосунок працюють із цілісною предметною моделлю. Нижче наведено сутності, знайдені у SQL-файлі actual5.sql.

№	Таблиця	Призначення у системі
1	achievements	досягнення
2	announcements	оголошення користувачам
3	attendance	облік відвідуваності
4	course_materials	навчальні матеріали
5	courses	навчальні курси
6	departments	кафедри або структурні підрозділи
7	grades	оцінки і результати перевірки
8	groups	навчальні групи
9	moderators	модератори системи
10	notifications	системні повідомлення
11	read_announcements	позначки прочитаних оголошень
12	schedule	розклад занять
13	student_achievements	зв'язок здобувачів із досягненнями
14	student_courses	зв'язок здобувачів із курсами
15	students	дані здобувачів
16	submissions	здані здобувачами роботи
17	tasks	завдання курсів
18	teacher_courses	зв'язок викладачів із курсами
19	teachers	дані викладачів
20	users	зберігання облікових записів користувачів і ролей

Enum-типи, знайдені у базі даних: lesson_format_enum, lesson_type_enum, material_category, user_role.

ДОДАТОК К

Код автоматизованих перевірок вебзастосунку

Нижче наведено приклади коду, який можна використати для демонстрації того, що тестування сайту було не лише описано в записці, а й підготовлено у вигляді відтворюваних автоматизованих перевірок.

Файл tests/web/mooden.spec.js:

```
import { test, expect } from '@playwright/test';

const pages = [
  'index.html',
  'student-dashboard.html',
  'student-courses.html',
  'student-tasks.html',
  'student-grades.html',
  'student-schedule.html',
  'teacher-dashboard.html',
  'teacher-courses.html',
  'teacher-submissions.html',
  'teacher-schedule.html'
];

test.describe('Mooden web smoke testing', () => {
  for (const pageName of pages) {
    test(`page ${pageName} opens without critical errors`, async ({ page }) => {
      const errors = [];
      page.on('pageerror', error => errors.push(error.message));
      await page.goto(`/frontend/${pageName}`);
      await expect(page.locator('body')).toBeVisible();
    });
  }
});
```

```

        await expect(page.locator('body')).not.toBeEmpty();
        expect(errors).toEqual([]);
    });
}
});

test('login page contains required controls', async ({ page }) => {
    await page.goto('/frontend/index.html');
    await expect(page.locator('input[type="email"]',
input[name="email"])).toBeVisible();
    await expect(page.locator('input[type="password"]',
input[name="password"])).toBeVisible();
    await expect(page.locator('button')).toBeVisible();
});

```

Файл tests/web/endpoint_audit.py:

```

from pathlib import Path
import re

```

```

ROOT = Path(__file__).resolve().parents[2]
FRONTEND = ROOT / 'frontend'
BACKEND = ROOT / 'backend'

```

```

fetch_pattern = re.compile(r"fetch\[([^\]]+)\]")
route_pattern = re.compile(r"router\.(get|post|put|patch|delete)\([^\)]+\)")

```

```

frontend_endpoints = set()
for file in FRONTEND.rglob('*.js'):
    text = file.read_text(encoding='utf-8', errors='ignore')
    frontend_endpoints.update(fetch_pattern.findall(text))

```

```

backend_routes = set()
for file in BACKEND.rglob('*.js'):
    text = file.read_text(encoding='utf-8', errors='ignore')
    for method, path in route_pattern.findall(text):
        backend_routes.add((method.upper(), path))

print('Frontend endpoints:', len(frontend_endpoints))
print('Backend routes:', len(backend_routes))
for endpoint in sorted(frontend_endpoints):
    print(endpoint)

```

Команди для запуску тестів у локальному середовищі:

```
npm install
```

```
npx playwright install
```

```
npx playwright test
```

```
python tests/web/endpoint_audit.py
```

ДОДАТОК Л

Таблиця функціональних тест-кейсів вебзастосунку Mooden

ID	Модуль	Сценарій	Очікуваний результат	Фактичний результат	Статус
WT-01	Авторизація	Відкриття стартової сторінки	Відображається форма входу, статистика та перемикачі інтерфейсу	index.html присутній, підключені CSS/JS файли знайдені, відсутніх ресурсів не виявлено	Passed
WT-02	Авторизація	Вхід з валідними даними здобувача	Токен зберігається, користувач переходить на student-dashboard.html	Логіка збереження token/userRole і маршрутизації реалізована у login-script.js	Passed
WT-03	Авторизація	Вхід з валідними даними викладача	Токен зберігається, користувач переходить на teacher-dashboard.html	Логіка рольового переходу реалізована у login-script.js	Passed
WT-04	Авторизація	Вхід з роллю moderator	Користувач переходить на moderator-dashboard.html	Маршрут для moderator-dashboard.html присутній	Passed
WT-05	Авторизація	Некоректний логін або пароль	Система показує повідомлення про помилку без переходу в кабінет	Обробка помилки відповіді API реалізована у login-script.js	Passed
WT-06	Авторизація	Захист сторінок без токена	Користувач перенаправляється на index.html	auth-guard.js перевіряє token/userRole перед доступом	Passed
WT-07	Інтерфейс	Перемикання теми	Тема змінюється і зберігається між відкриттями	theme-manager.js використовує mooden-theme у localStorage	Passed
WT-08	Інтерфейс	Перемикання мови	Текст інтерфейсу змінюється українська/англійська	i18n-manager.js і translations.js підключені	Passed
WT-09	Інтерфейс	Показ/приховування пароля	Тип поля password/text перемикається	toggle-password.js підключений до стартової сторінки	Passed
WT-10	Здобувач	Відкриття здобувацької панелі	Відображаються зведені показники, оголошення та дані навчання	GET /api/student/dashboard має відповідний backend-маршрут	Passed
WT-11	Здобувач	Перегляд курсів здобувача	Відображається список доступних курсів	GET /api/student/courses зіставлено з backend-маршрутом	Passed
WT-12	Здобувач	Відкриття деталей курсу	Відображаються матеріали, завдання та інформація курсу	GET /api/student/courses/:id/detail зіставлено з backend-маршрутом	Passed
WT-13	Здобувач	Перегляд списку завдань	Відображається список завдань із дедлайнами	GET /api/student/tasks зіставлено з backend-маршрутом	Passed
WT-14	Здобувач	Відкриття деталей завдання	Відображається опис завдання, дедлайн і блок задачі	GET /api/student/tasks/:id зіставлено з backend-маршрутом	Passed
WT-15	Здобувач	Надсилання виконаного завдання	Файл передається на сервер через multipart/form-data	POST /api/student/tasks/:id/submit реалізовано з multer middleware	Passed
WT-16	Здобувач	Перегляд оцінок	Відображаються оцінки та коментарі	GET /api/student/grades зіставлено з backend-маршрутом	Passed

ID	Модуль	Сценарій	Очікуваний результат	Фактичний результат	Статус
WT-17	Здобувач	Перегляд розкладу	Відображаються заняття за вибраний період	GET /api/student/schedule зіставлено з backend-маршрутом	Passed
WT-18	Здобувач	Відмітка відвідуваності	На сервер відправляється запит позначення відвідуваності	POST /api/student/attendance/mark зіставлено з backend-маршрутом	Passed
WT-19	Здобувач	Перегляд відвідуваності курсу	Відображається статистика відвідуваності	GET /api/student/courses/:id/attendance зіставлено з backend-маршрутом	Passed
WT-20	Здобувач	Перегляд профілю	Відображаються персональні дані здобувача	GET /api/student/profile зіставлено з backend-маршрутом	Passed
WT-21	Викладач	Відкриття викладацької панелі	Відображаються курси, активність та оголошення	GET /api/teacher/dashboard зіставлено з backend-маршрутом	Passed
WT-22	Викладач	Перегляд курсів викладача	Відображається список курсів викладача	GET /api/teacher/courses зіставлено з backend-маршрутом	Passed
WT-23	Викладач	Деталі курсу викладача	Відображаються матеріали, учасники та завдання курсу	GET /api/teacher/courses/:id/detail зіставлено з backend-маршрутом	Passed
WT-24	Викладач	Перегляд робіт на перевірку	Відображається список зданих робіт	GET /api/teacher/submissions зіставлено з backend-маршрутом	Passed
WT-25	Викладач	Оцінювання роботи	Оцінка зберігається на сервері	POST /api/teacher/submissions/:id/grade зіставлено з backend-маршрутом	Passed
WT-26	Викладач	Перегляд розкладу викладача	Відображається розклад за обраний період	GET /api/teacher/schedule зіставлено з backend-маршрутом	Passed
WT-27	Викладач	Перегляд профілю викладача	Відображаються персональні дані викладача	GET /api/teacher/profile зіставлено з backend-маршрутом	Passed
WT-28	Користувач	Відображення sidebar	Sidebar завантажує ім'я, роль і навігацію	GET /api/user/sidebar зіставлено з backend-маршрутом	Passed
WT-29	Користувач	Зміна пароля	Сервер приймає запит і повертає результат	POST /api/user/change-password зіставлено з backend-маршрутом	Passed
WT-30	Користувач	Перегляд сповіщень	Відображається список сповіщень	GET /api/user/notifications зіставлено з backend-маршрутом	Passed
WT-31	Користувач	Позначення сповіщення прочитаним	Відправляється POST /api/user/notifications/:id/read	Маршрут присутній у backend	Passed
WT-32	Користувач	Позначення всіх сповіщень прочитаними	Відправляється POST /api/user/notifications/read-all	Маршрут присутній у backend	Passed
WT-33	Користувач	Перегляд оголошень	Відображається список оголошень	GET /api/user/announcements зіставлено з backend-маршрутом	Passed

ID	Модуль	Сценарій	Очікуваний результат	Фактичний результат	Статус
WT-34	Користувач	Позначення оголошення прочитаним	Відправляється POST /api/user/announcements/:id/read	Маршрут присутній у backend	Passed
WT-35	Користувач	Збереження налаштувань мови	Відправляється PATCH /api/user/settings	Маршрут присутній у backend	Passed
WT-36	Модератор	Відкриття панелі модератора	Відображається moderator-dashboard.html	Сторінка та скрипт moderator-dashboard.js присутні	Passed
WT-37	API	Відповідність frontend-запитів backend-маршрутам	Усі запити мають backend-обробник	30/30 унікальних frontend endpoint patterns зіставлено	Passed
WT-38	Статика	Перевірка HTML/CSS/JS посилань	Відсутні биті локальні посилання	Перевірено 21 HTML-сторінку і 387 локальних посилань, помилок 0	Passed
WT-39	Код	Синтаксична перевірка JavaScript	Синтаксичних помилок немає	Перевірка пройдена для frontend/js і backend/src	Passed
WT-40	База даних	Наявність структури БД	SQL dump містить таблиці та enum-типи для доменної моделі	Знайдено 20 таблиць і 4 enum-типи	Passed
WT-41	Безпека	Передача JWT у запитах	Захищені запити містять Authorization Bearer token	У JS-модулях використовується token із localStorage/sessionStorage	Passed
WT-42	Обробка помилок	Реакція на 401	Токен очищається, користувач повертається на сторінку входу	У модулях кабінетів реалізовано очистку сховища і redirect на index.html	Passed
WT-43	Файли	Завантаження здобувацької роботи	Файл приймається backend і зберігається в uploads	submitTask використовує uploadSubmission.single("file")	Passed

ДОДАТОК М

Фрагменти коду автоматизованого тестування вебзастосунку

Приклад Playwright-тесту для перевірки стартової сторінки та доступності ключових елементів інтерфейсу:

```
import { test, expect } from '@playwright/test';

test('відкриття сторінки авторизації Mooden', async ({ page }) => {
  await page.goto('frontend/index.html');
  await expect(page.locator('input[type="email"]')).toBeVisible();
  await expect(page.locator('input[type="password"]')).toBeVisible();
  await expect(page.locator('button')).toContainText(/увійти|login|sign/i);
});
```

Приклад Python-фрагмента для аудиту відповідності frontend API-запитів backend-маршрутам:

```
from pathlib import Path
import re

frontend_calls = set()
for js_file in Path('frontend').rglob('*.js'):
    text = js_file.read_text(encoding='utf-8', errors='ignore')
    frontend_calls.update(re.findall(r'fetch\(["']([^\"]+)\)', text))

backend_routes = set()
for route_file in Path('backend').rglob('*.js'):
    text = route_file.read_text(encoding='utf-8', errors='ignore')
    backend_routes.update(re.findall(r'router\.(get|post|put|delete)\(["']([^\"]+)\)', text))

print('Frontend API calls:', len(frontend_calls))
print('Backend routes:', len(backend_routes))
```

ДОДАТОК Н

Звіт результатів перевірки на унікальність тексту в базі ХНУРЕ





Звіт не був оцінений



Метадані


ДОКУМЕНТ

Заголовок

2020_Б_ПІ_ККП_ПЗПІ_23_7_Макошменко_М_В_вкорочений

Автор

Макошменко_М_В

Науковий керівник / Експерт

Олійник О.В. / Копоть М.А.

ІД документу

334177820


ОРГАНІЗАЦІЯ

Назва організації

Kharkiv National University of Radio Electronics

підрозділ

каф. ПІ


ЗВІТ

Дата звіту

04/2020

Дата редагування

Обсяг знайдених подібностей

Коефіцієнт подібності означає, який відсоток тексту по відношенню до загального обсягу тексту було знайдено в різних джерелах. Зверніть увагу, що високі значення коефіцієнта не автоматично означають плагиат. Звіт має аналізувати компетентна / уповноважена особа.

0%

0%

КП 1

5328

Кількість слів

0.13%

0.13%

КЦ

42512

Кількість символів

Тривога

У цьому розділі ви знайдете інформацію щодо текстових спотворень. Ці спотворення в тексті можуть говорити про МІСКОПІВІ маніпуляції в тексті. Спотворення в тексті можуть мати намісний характер, але частіше характер технічних помилок при конвертації документа та його збереженні, тому ми рекомендуємо вам підходити до аналізу цього модуля відповідально. У разі виявлення зауважень, просимо звертатися до нашої служби підтримки.

Заміна букв		0
Інтервали		0
Мікропробіли		0
Білі знаки		0
Парафрази (SmartMarks)		0

Джерела

Нижче наведений список джерел. В цьому списку є джерела із різних баз даних. Колір тексту означає в якому джерелі він був знайдений. Ці джерела і значення Коефіцієнту Подібності не відображають прямого плагиату. Необхідно відкрити кожне джерело і проаналізувати зміст і правильність оформлення джерела.

10 найдовших фраз

Колір тексту

#	НАЗВА ТА АДРЕСА ДЖЕРЕЛА URL (НАЗВА БАЗИ)	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
	База даних RefBooks	

ДОДАТОК П

Слайди презентації

Освітня платформа для організації віртуального навчального процесу

Функціональне тестування мобільного та вебзастосунку

Виконав: ПЗПІ-23-7, Максименко Марко Віталійович

Керівник: ст. викл. кафедри ПІ, Олійник Олена Володимирівна

2026 р.

1

Загальна ідея платформи Mooden

Спільна частина командного проекту

єдине середовище для організації віртуального навчального процесу;
доступ до курсів, матеріалів, завдань, тестів і оцінок з мобільного та вебклієнта;
рольова модель: здобувач, викладач, адміністратор;
робота через спільний backend і централізовану базу даних.

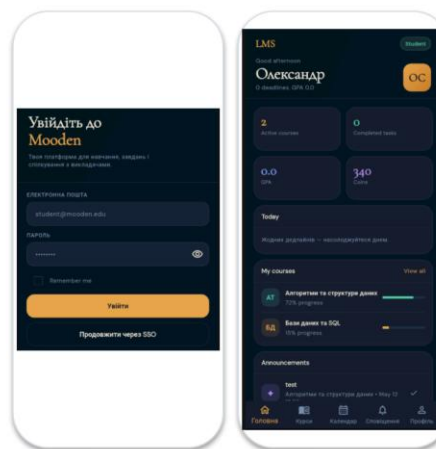


Рисунок 1 – Головні екрани мобільної частини

2

Актуальність розробки освітньої платформи

Проблеми, які вирішує командний проєкт

1. Мобільний доступ

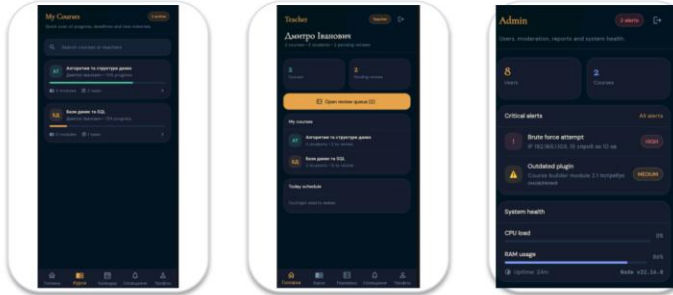
Студент повинен швидко отримувати завдання, новини та оцінки зі смартфона.

2. Єдині дані

Веб і mobile повинні працювати з одними ролями, курсами та backend-маршрутами.

3. Контроль якості

Платформа має стабільно працювати без помилок авторизації та навігації.



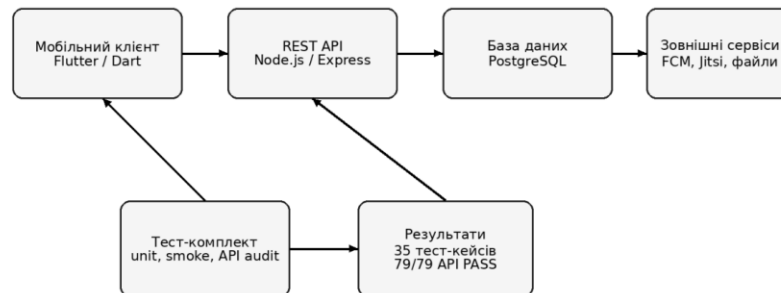
Функціональне тестування потрібне для підтвердження того, що ключові сценарії користувача виконуються коректно і не втрачають дані при взаємодії з API.

3

Архітектура тестованої системи Mooden

Клієнтські частини працюють через єдину серверну логіку

Архітектура тестованої системи Mooden



Під час тестування перевірявся ланцюг mobile / web → REST API → PostgreSQL → результати в інтерфейсі.

4

Ролі користувачів і основні модулі

Діаграма варіантів використання для здобувача, викладача та адміністратора

На діаграмі зображено основні сценарії роботи з платформою для трьох ролей користувачів.

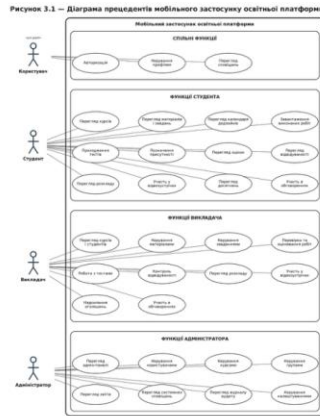


Рисунок 2 – Варіанти використання освітньої платформи для різних ролей

5

Структура даних платформи

Окремо винесена діаграма для читабельності

схема даних відображає зв'язки між користувачами, курсами, завданнями та оцінками;

ця діаграма використовувалася для формування тестів доступу та перевірки цілісності даних.

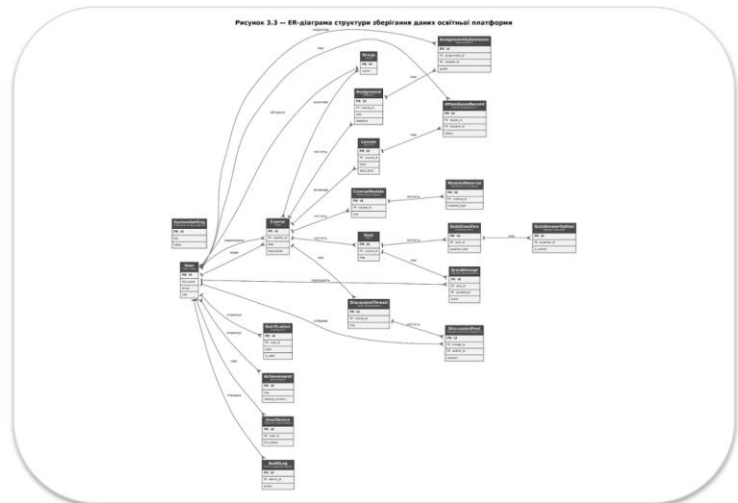


Рисунок 3 – Фрагмент схеми даних платформи Moodle

6

Індивідуальна частина: функціональне тестування

Завдання моєї роботи у межах ККП

перевірити працездатність основних сценаріїв mobile та web;
сформувані тест-кейси з кроками та очікуваним результатом;
провести аудит відповідності frontend API-запитів backend-маршрутам;
додати приклади автоматизованих перевірок для подальшого
регресійного тестування.

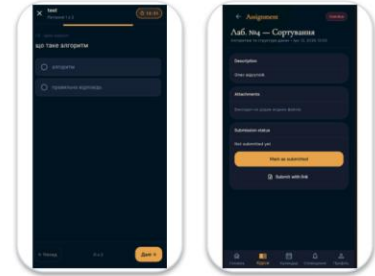


Рисунок 4 – Екрани, використані в перевірях

7

Процес функціонального тестування

Послідовність виконання індивідуальної частини

Процес функціонального тестування



Результатом цього процесу стали таблиці тестів, таблиці результатів та приклади автоперевірок.

8

Середовище та інструменти тестування

Компоненти, що використовувалися під час перевірок

1 Flutter / Dart

Мобільний клієнт, маршрути, віджети, smoke- та unit-перевірки.

2 HTML / CSS / JS

Вебчастина, сторінки кабінетів, навігація, локальні ресурси.

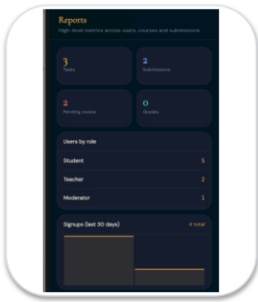
3 Node.js / Express

REST API, JWT, ролі, backend-маршрути, бізнес-логіка.

4 PostgreSQL / Playwright

База даних, API audit, web-перевірки й фіксація результатів.

ручні функціональні тест-кейси — для перевірки користувацьких сценаріїв;
unit і smoke-тести — для локальної валідації коду;
Playwright — для перевірки вебсторінок;
Python API audit — для звірки маршрутів клієнта та backend.



Фрагмент таблиці функціональних тестів

Приклад структури ручного тест-кейсу

ID	Сценарій	Очікуваний результат	Фактичний результат	Статус
M-01	Вхід студента з валідними даними	Відкрито головний екран	Головний екран відкрито	PASS
M-04	Перехід до списку курсів	Показано доступні курси	Курси відображено	PASS
M-09	Відкриття завдання	Показано опис і вкладки	Дані завдання завантажено	PASS
W-03	Відкриття сторінки users	Таблиця користувачів доступна	Сторінка працює	PASS
API-07	Зіставлення POST /login	Маршрут наявний на backend	Маршрут знайдено	PASS

У повному тест-комплекті використовувалися окремі набори сценаріїв для mobile, web та технічної перевірки API.

Таблиця результатів мобільного тестування

Ключові сценарії студентського мобільного інтерфейсу

Перевірка	Кількість / статус
Тест-кейси mobile	35 / PASS
Успішні сценарії	35 / PASS
Критичні дефекти	0
Перевірені модулі	авторизація, курси, завдання, тести, оцінки
Середовище	Flutter-клієнт + backend API

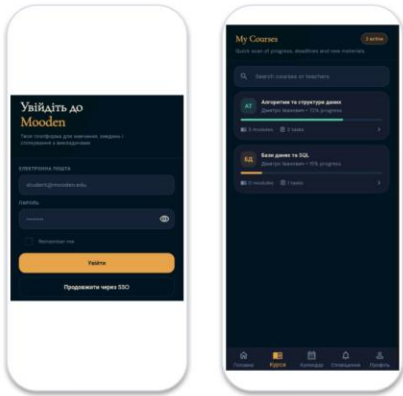


Рисунок 5 – Перевірка входу та списку курсів

11

Таблиця результатів вебтестування

Ключові сценарії для ролей викладача та адміністратора

Перевірка	Кількість / статус
Тест-кейси web	43 / PASS
Успішні сценарії	43 / PASS
Критичні дефекти	0
Перевірені модулі	users, reports, role pages, локальні ресурси
Середовище	Web-клієнт + backend API

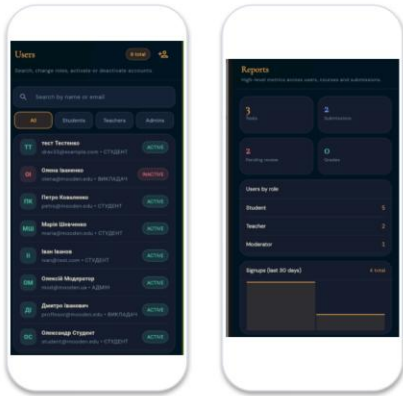
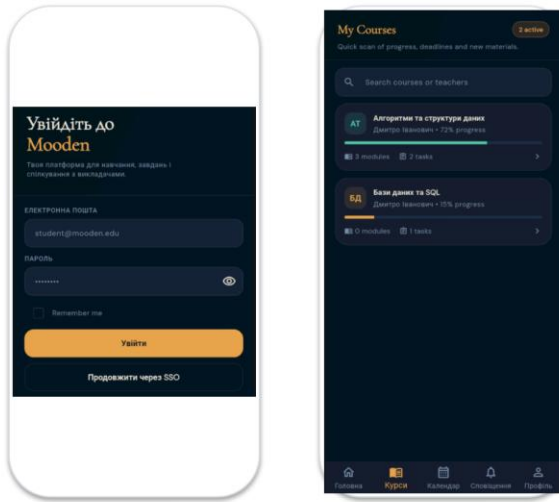


Рисунок 6 – Приклади перевірених вебекранів

12

Приклад сценарію: авторизація та перегляд курсів

Великі читабельні скриншоти інтерфейсу

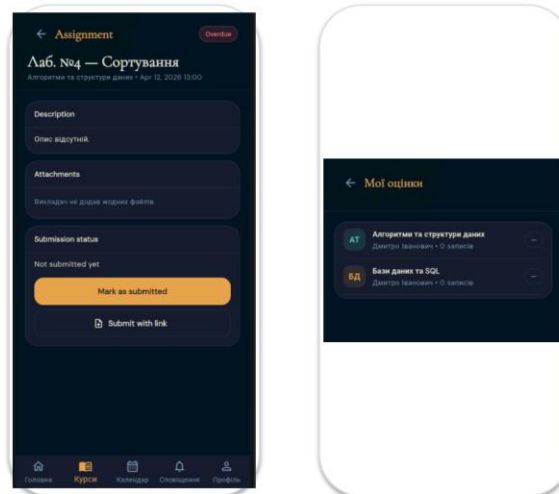


користувач вводить валідні облікові дані;
після входу відкривається список доступних курсів;
перевіряється коректність навігації та відображення
даних.

13

Приклад сценарію: перегляд завдання та оцінок

Перевірка навчального контенту та результатів

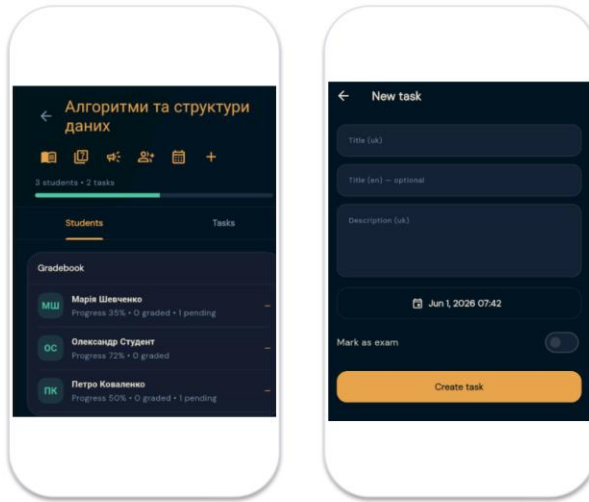


відкривається конкретне завдання з описом і
вкладеннями;
перевіряється подання роботи та зміна статусу;
оцінки мають відповідати даним, отриманим із
backend.

14

Приклад сценарію: викладач створює завдання

Перевірка ролі викладача та форми введення



викладач переходить до курсу і відкриває форму створення завдання;

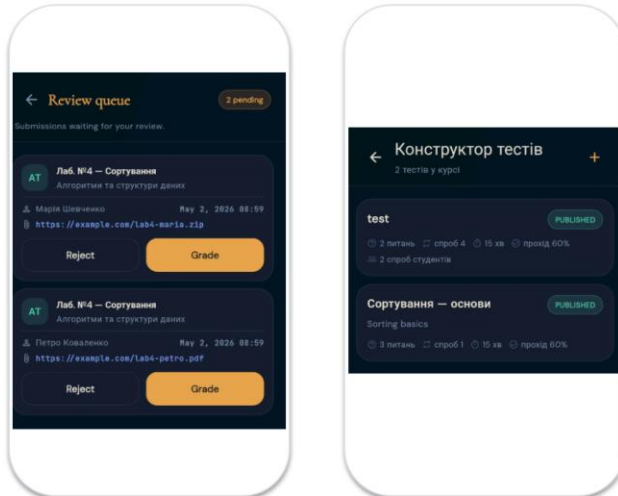
поля назви, опису та дедлайну мають бути доступними;

після збереження нове завдання повинне відображатися у курсі.

15

Приклад сценарію: review queue та конструктор тестів

Перевірка складніших інтерфейсних сценаріїв



перевіряється відображення робіт, що очікують оцінювання;

викладач має можливість перейти до перевірки та виставлення оцінки;

у конструкторі тестів контролюється створення та відображення питань.

16

Фрагмент коду: unit-тест ролей

Локальна перевірка логіки доступу

```
dart / unit-test

test('student may open courses tab', () async {
  final result = canOpenTab(role: 'student', tab: 'courses');
  expect(result, true);
});

test('guest may not open grade page', () async {
  final result = canOpenPage(role: 'guest', page: 'grades');
  expect(result, false);
});
```

перевіряється дозволений доступ до елементів інтерфейсу;

логіка ролей контролюється ще до ручного тестування;

такі тести зручно запускати локально після кожної зміни.



17

Фрагменти коду: smoke-тест і Playwright

Приклади автоматизації для mobile та web

```
Flutter smoke-test

testWidgets('login screen is visible', (tester) async {
  await tester.pumpWidget(const MyApp());
  expect(find.text('Увійти'), findsOneWidget);
  await tester.enterText(find.byType(TextField).first, 'user@test.com');
  await tester.tap(find.text('Увійти'));
  await tester.pump();
});
```

```
Playwright web-test

test('student dashboard loads', async ({ page }) => {
  await page.goto('/student');
  await expect(page.getByText('My Courses')).toBeVisible();
  await expect(page.getByText('Assignments')).toBeVisible();
});
```

Smoke-тест підтверджує базовий запуск застосунку, а Playwright — доступність ключових елементів вебінтерфейсу.

18

Результати аудиту API та узагальнення тестування

Підсумок технічних і функціональних перевірок

```
Python API audit
for route in frontend_routes:
    key = (route['method'], route['path'])
    if key in backend_routes:
        passed += 1
    else:
        mismatches.append(route)
print(f'PASS: {passed}')
```

Показник	Значення
Перевірені API-запити	79
Відповідні backend routes	79
Невідповідності	0
Статус	PASS

35

mobile

43

web

0

збоїв

Ручне та автоматизоване тестування підтвердило коректність основних сценаріїв користувача. Критичних відмов або невідповідностей між frontend і backend не виявлено.

Висновки

Отримані результати та подальший розвиток

- сформовано набір функціональних тест-кейсів для mobile та web-частини системи;
- виконано перевірку основних сценаріїв: авторизація, курси, завдання, тести, ролі користувачів;
- проведено аудит API та підтверджено відповідність маршрутів frontend і backend;
- підготовлено приклади автотестів, які можна використовувати для регресійної перевірки.

Практичний результат

Презентація показує власний внесок у ККП: тестові таблиці, результати, код перевірок і великі скриншоти ключових сценаріїв.

Подальший розвиток

Надалі набір можна розширити e2e-сценаріями, негативними перевітками та автоматичним запуском у CI.

Результати роботи підтверджують готовність тестового комплекту до демонстрації під час захисту ККП.